

```
# =====
# CELL 38E – CUDA DEBUG MODE
# MUST BE RUN FIRST AFTER RUNTIME RESTART
# =====

import os

os.environ["CUDA_LAUNCH_BLOCKING"] = "1"

print("CUDA_LAUNCH_BLOCKING =", os.environ["CUDA_LAUNCH_BLOCKING"])

import torch

print("PyTorch version :", torch.__version__)
print("CUDA available :", torch.cuda.is_available())
print("GPU      :", torch.cuda.get_device_name(0))
print("CUDA version  :", torch.version.cuda)

# Basic CPU → GPU transfer
x = torch.tensor([0, 1, 2, 3], dtype=torch.long)

print("\nCPU tensor:", x)

x_gpu = x.to("cuda")

print("GPU tensor:", x_gpu)

# Basic embedding
embedding = torch.nn.Embedding(
    num_embeddings=8000,
    embedding_dim=256
).cuda()

y = embedding(x_gpu)

print("Embedding shape:", tuple(y.shape))

print("\n" + "=" * 70)
print("PASS – CUDA DEBUG MODE INITIALIZED SUCCESSFULLY")
print("=" * 70)
```

```
CUDA_LAUNCH_BLOCKING = 1
PyTorch version : 2.11.0+cu128
CUDA available  : True
GPU             : Tesla T4
CUDA version    : 12.8

CPU tensor: tensor([0, 1, 2, 3])
GPU tensor: tensor([0, 1, 2, 3], device='cuda:0')
Embedding shape: (4, 256)

=====
PASS – CUDA DEBUG MODE INITIALIZED SUCCESSFULLY
=====
```

```
from google.colab import drive
from pathlib import Path
import os

# Check if Google Drive is already mounted
if not os.path.exists('/content/drive/MyDrive'):
    print("Mounting Google Drive...")
    drive.mount('/content/drive')
else:
    print("Google Drive is already mounted.")
```

```
Mounting Google Drive...
Mounted at /content/drive
```

```
# =====
# CELL 1: EXPERIMENT CONFIGURATION
# =====

import json
import random
import unicodedata
from pathlib import Path
from collections import defaultdict
from typing import List, Tuple, Dict, Set, Optional

import pandas as pd
import matplotlib.pyplot as plt
```

```

# -----
# Reproducibility
# -----

RANDOM_SEED = 42
random.seed(RANDOM_SEED)

# -----
# Paths
# -----

# BASE_DIR = Path("/content")

BASE_DIR = Path(
    "/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment"
)

MODEL_DIR = BASE_DIR / "models"
RESULT_DIR = BASE_DIR / "results"

DATA_PATH = BASE_DIR / "igbo_morph_scoped_corpus.json"

for d in [MODEL_DIR, RESULT_DIR]:
    d.mkdir(parents=True, exist_ok=True)

print("Environment ready at:", BASE_DIR)

# -----
# Experimental conditions
# -----

TARGET_VOCAB_SIZES = [2000, 4000, 8000, 12000, 16000]

TRAIN_RATIO = 0.90
EVAL_SAMPLE_SIZE = 10_000

print("Experiment configuration")
print("-" * 50)
print(f"Random seed:      {RANDOM_SEED}")
print(f"Training ratio:    {TRAIN_RATIO}")
print(f"Evaluation sample: {EVAL_SAMPLE_SIZE}")
print(f"Vocabulary sizes:  {TARGET_VOCAB_SIZES}")
print(f>Data path:        {DATA_PATH}")
print(f"Model directory:   {MODEL_DIR}")
print(f"Results directory: {RESULT_DIR}")

```

```

Environment ready at: /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment
Experiment configuration
-----
Random seed:      42
Training ratio:    0.9
Evaluation sample: 10000
Vocabulary sizes: [2000, 4000, 8000, 12000, 16000]
Data path:        /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/igbo_morph_scoped_corpus.json
Model directory:   /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models
Results directory: /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/results

```

```

# =====
# CELL 2: LOAD MORPHOLOGICALLY ANNOTATED CORPUS
# =====

with open(DATA_PATH, "r", encoding="utf-8") as f:
    raw_corpus_data = json.load(f)

print("Corpus loaded successfully.")
print(f"Number of word instances: {len(raw_corpus_data):,}")

print("\nExample entry:")
print(raw_corpus_data[0])

```

```

Corpus loaded successfully.
Number of word instances: 833,843

Example entry:
[['S', 'o', 'w', 'o', 'r', 'e']]

```

```

# =====
# CELL 3: DEFINE ORTHOGRAPHIC UNITIZATION
# =====

IGBO_DIGRAPHS = {
    "ch", "gb", "gh", "gw", "kp", "kw", "nw", "ny", "sh",
    "Ch", "Gb", "Gh", "Gw", "Kp", "Kw", "Nw", "Ny", "Sh",

```

```

    "CH", "GB", "GH", "GW", "KP", "KW", "NW", "NY", "SH"
}

```

```

def normalize_nfc(text: str) -> str:
    """
    Normalize text using Unicode NFC.
    """
    return unicodedata.normalize("NFC", text)

def unitize_morpheme(morpheme: str) -> Tuple[str, ...]:
    """
    Convert a morpheme into orthographic units.

    Igbo digraphs are treated as single units.
    All other characters are treated as individual units.
    """
    norm_m = normalize_nfc(morpheme)

    units = []
    i = 0
    n = len(norm_m)

    while i < n:
        if i + 1 < n and norm_m[i:i+2] in IGBO_DIGRAPHS:
            units.append(norm_m[i:i+2])
            i += 2
        else:
            units.append(norm_m[i])
            i += 1

    return tuple(units)

print("Orthographic unitization defined.")

example = "anyị"
print(f"Example: {example}")
print(f"Units: {unitize_morpheme(example)}")

```

```

Orthographic unitization defined.
Example: anyị
Units: ('a', 'ny', 'i')

```

Start coding or [generate](#) with AI.

```

# =====
# CELL 4: CREATE TRAINING AND EVALUATION SPLIT
# =====

shuffled_data = list(raw_corpus_data)
random.shuffle(shuffled_data)

split_idx = int(TRAIN_RATIO * len(shuffled_data))

train_raw = shuffled_data[:split_idx]
eval_raw = shuffled_data[split_idx:]

# Fixed evaluation sample for computational consistency
eval_test_sample = eval_raw[:EVAL_SAMPLE_SIZE]

print("Experimental split")
print("-" * 50)
print(f"Total instances:      {len(raw_corpus_data):,}")
print(f"Training instances:     {len(train_raw):,}")
print(f"Evaluation instances:   {len(eval_raw):,}")
print(f"Evaluation sample:      {len(eval_test_sample):,}")
print(f"Training proportion:    {len(train_raw)/len(raw_corpus_data):.2%}")

```

```

Experimental split
-----
Total instances:      833,843
Training instances:   750,458
Evaluation instances:  83,385
Evaluation sample:    10,000
Training proportion:   90.00%

```

```

# =====
# CELL 5: BUILD MORPHOLOGY-AWARE AND BASELINE BPE CORPORA
# =====

from collections import defaultdict
from typing import Dict, List, Tuple

```

```

# -----
# Morphologically-aware representation
#
# Input schema:
# [
#     ['n', 'w', 'u'],      # morpheme 1
#     ['c', 'h', 'i'],      # morpheme 2
#     ['e']                  # morpheme 3
# ]
#
# Output:
# (
#     ('nw', 'u'),          # morpheme 1
#     ('ch', 'i'),          # morpheme 2
#     ('e',)                # morpheme 3
# )
#
# The outer tuple preserves morphological boundaries.
# -----

def clean_morpheme_chars(chars: List[str]) -> str:
    """
    Convert one annotated morpheme from character-list form
    into a normalized surface string.

    Whitespace characters are removed because they are
    orthographic separators, not morphological material.
    Other characters, including punctuation, are retained.
    """
    text = "".join(chars)

    # Remove whitespace only; retain punctuation and letters.
    text = "".join(ch for ch in text if not ch.isspace())

    return normalize_nfc(text)

def build_morph_frequency_dict(
    raw_data
) -> Dict[Tuple[Tuple[str, ...], ...], int]:
    """
    Build the morphology-aware BPE training corpus.

    Each input instance is:
        list[morpheme]
    where each morpheme is:
        list[character]

    The resulting WordTuple retains the morphological
    boundaries between morphemes.
    """
    corpus_freqs = defaultdict(int)

    for word_morphemes in raw_data:
        if not word_morphemes:
            continue

        morpheme_units = []

        for chars in word_morphemes:
            if not chars:
                continue

            morpheme = clean_morpheme_chars(chars)

            # Ignore empty morphemes created entirely from whitespace.
            if not morpheme:
                continue

            units = unitize_morpheme(morpheme)

            if units:
                morpheme_units.append(units)

        if morpheme_units:
            word_tuple = tuple(morpheme_units)
            corpus_freqs[word_tuple] += 1

    return dict(corpus_freqs)

def build_baseline_frequency_dict(
    raw_data
) -> Dict[Tuple[Tuple[str, ...], ...], int]:

```

```

"""
Build the unsegmented baseline BPE corpus.

The same scoped word instances are used, but morphological
boundaries are flattened before BPE training.

Thus:
[['a'], ['g', 'a']]

becomes:
(('a', 'g', 'a'),)
"""
corpus_freqs = defaultdict(int)

for word_morphemes in raw_data:

    if not word_morphemes:
        continue

    all_units = []

    for chars in word_morphemes:

        if not chars:
            continue

        morpheme = clean_morpheme_chars(chars)

        if not morpheme:
            continue

        all_units.extend(unitize_morpheme(morpheme))

    if all_units:
        corpus_freqs[(tuple(all_units),)] += 1

return dict(corpus_freqs)

# -----
# Build both training representations from the SAME scoped data
# -----

morph_corpus = build_morph_frequency_dict(train_raw)
baseline_corpus = build_baseline_frequency_dict(train_raw)

print("BPE training corpora constructed")
print("-" * 60)
print(f"MorphBPE types:  {len(morph_corpus):,}")
print(f"Baseline types:  {len(baseline_corpus):,}")

```

```

BPE training corpora constructed
-----

```

```

MorphBPE types:  42,754
Baseline types:  42,497

```

```

# =====
# CELL 6: VALIDATE BPE CORPUS REPRESENTATIONS
# =====

def find_whitespace_units(corpus):
    found = set()

    for word_tuple in corpus:
        for morpheme in word_tuple:
            for unit in morpheme:
                if any(ch.isspace() for ch in unit):
                    found.add(unit)

    return found

def count_morpheme_structures(corpus):
    """
    Count how many corpus types contain multiple morphemes.
    """
    single = 0
    multi = 0

    for word_tuple in corpus:
        if len(word_tuple) == 1:
            single += 1
        else:
            multi += 1

```

```

return single, multi

morph_whitespace = find_whitespace_units(morph_corpus)
baseline_whitespace = find_whitespace_units(baseline_corpus)

morph_single, morph_multi = count_morpheme_structures(morph_corpus)

print("Corpus validation")
print("=" * 60)

print(f"MorphBPE types:           {len(morph_corpus):,}")
print(f"Baseline types:           {len(baseline_corpus):,}")

print(f"\nMorphBPE whitespace units:  {morph_whitespace}")
print(f"Baseline whitespace units:  {baseline_whitespace}")

print(f"\nMorphBPE single-morpheme:    {morph_single:,}")
print(f"MorphBPE multi-morpheme:     {morph_multi:,}")

```

```

Corpus validation
=====
MorphBPE types:           42,754
Baseline types:           42,497

MorphBPE whitespace units:  set()
Baseline whitespace units:  set()

MorphBPE single-morpheme:   22,843
MorphBPE multi-morpheme:   19,911

```

```

# =====
# CELL 7: INSPECT RECONSTRUCTED MORPHOLOGICAL STRUCTURE
# =====

print("Sample MorphBPE representations")
print("=" * 60)

shown = 0

for word_tuple, freq in morph_corpus.items():

    if len(word_tuple) > 1:

        print(f"Frequency: {freq}")
        print("Morphemes:")
        for i, morpheme in enumerate(word_tuple):
            print(f"  {i}: {morpheme}")

        print("Surface:")
        print("".join("".join(m) for m in word_tuple))

        print("-" * 60)

        shown += 1

        if shown >= 10:
            break

```

```

Surface:
ga-amasi
-----
Frequency: 515
Morphemes:
  0: ('kw', 'u')
  1: ('o',)
Surface:
kwuo

```

```

1: ('r', 'q')
Surface:
chqrq
-----
Frequency: 55
Morphemes:
0: ('a',)
1: ('m', 'a', 'l', 'i', 't', 'e')
2: ('l', 'a')
Surface:
amalitela
-----
Frequency: 1392
Morphemes:
0: ('b', 'u')
1: ('r', 'u')
Surface:
buru
-----
Frequency: 3
Morphemes:
0: ('i',)
1: ('kp', 'a')
2: ('ch', 'a')
3: ('r', 'a')
Surface:
ikpachara
-----

```

```

# =====
# CELL 8: VERIFY MORPH/BASELINE SURFACE EQUIVALENCE
# =====

def surface_from_word_tuple(word_tuple):
    return "".join(
        unit
        for morpheme in word_tuple
        for unit in morpheme
    )

morph_surface_counts = defaultdict(int)

for word_tuple, freq in morph_corpus.items():
    surface = surface_from_word_tuple(word_tuple)
    morph_surface_counts[surface] += freq

baseline_surface_counts = defaultdict(int)

for word_tuple, freq in baseline_corpus.items():
    surface = surface_from_word_tuple(word_tuple)
    baseline_surface_counts[surface] += freq

print("Surface-equivalence validation")
print("=" * 60)

print(
    "MorphBPE total instances:",
    f"{sum(morph_surface_counts.values()):,}"
)

print(
    "Baseline total instances:",
    f"{sum(baseline_surface_counts.values()):,}"
)

print(
    "MorphBPE unique surfaces:",
    f"{len(morph_surface_counts):,}"
)

print(
    "Baseline unique surfaces:",
    f"{len(baseline_surface_counts):,}"
)

print(
    "Identical surface inventories:",
    morph_surface_counts.keys() == baseline_surface_counts.keys()
)

print(
    "Identical surface frequencies:",
    morph_surface_counts == baseline_surface_counts
)

```

Surface-equivalence validation

```
=====
MorphBPE total instances: 750,457
Baseline total instances: 750,457
MorphBPE unique surfaces: 42,496
Baseline unique surfaces: 42,496
Identical surface inventories: True
Identical surface frequencies: True
```

```
# =====
# CELL 9: IDENTIFY SURFACE-EQUIVALENCE MISMATCHES
# =====

morph_only = set(morph_surface_counts) - set(baseline_surface_counts)
baseline_only = set(baseline_surface_counts) - set(morph_surface_counts)

frequency_mismatches = {
    surface: (
        morph_surface_counts[surface],
        baseline_surface_counts[surface]
    )
    for surface in morph_surface_counts.keys() & baseline_surface_counts.keys()
    if morph_surface_counts[surface] != baseline_surface_counts[surface]
}

print("Surface-equivalence mismatch analysis")
print("=" * 60)

print(f"Surfaces only in MorphBPE:    {len(morph_only):,}")
print(f"Surfaces only in Baseline:    {len(baseline_only):,}")
print(f"Frequency mismatches:          {len(frequency_mismatches):,}")

print("\nExamples: MorphBPE only")
for surface in list(morph_only)[:10]:
    print(repr(surface), morph_surface_counts[surface])

print("\nExamples: Baseline only")
for surface in list(baseline_only)[:10]:
    print(repr(surface), baseline_surface_counts[surface])

print("\nExamples: frequency mismatches")
for surface, counts in list(frequency_mismatches.items())[:10]:
    print(
        repr(surface),
        "MorphBPE =", counts[0],
        "Baseline =", counts[1]
    )
```

Surface-equivalence mismatch analysis

```
=====
Surfaces only in MorphBPE:    0
Surfaces only in Baseline:    0
Frequency mismatches:          0
```

Examples: MorphBPE only

Examples: Baseline only

Examples: frequency mismatches

```
# =====
# CELL 10: DIRECT INSTANCE-LEVEL SURFACE VALIDATION
# =====

def surface_from_raw_instance(word_morphemes):
    parts = []

    for chars in word_morphemes:
        if not chars:
            continue

        text = "".join(ch for ch in chars if not ch.isspace())
        text = normalize_nfc(text)

        if text:
            parts.append(text)

    return "".join(parts)

def morph_surface_from_instance(word_morphemes):
    morpheme_units = []

    for chars in word_morphemes:
        if not chars:
            continue
```



```

        morpheme = clean_morpheme_chars(chars)

        if not morpheme:
            continue

        units = unitize_morpheme(morpheme)

        if units:
            morpheme_units.append(units)

    word_tuple = tuple(morpheme_units)

    return surface_from_word_tuple(word_tuple)

def baseline_surface_from_instance(word_morphemes):
    all_units = []

    for chars in word_morphemes:
        if not chars:
            continue

        morpheme = clean_morpheme_chars(chars)

        if not morpheme:
            continue

        all_units.extend(unitize_morpheme(morpheme))

    word_tuple = (tuple(all_units),)

    return surface_from_word_tuple(word_tuple)

# -----
# Compare every raw training instance
# -----

raw_vs_morph_mismatches = []
raw_vs_baseline_mismatches = []
morph_vs_baseline_mismatches = []

for idx, word_morphemes in enumerate(train_raw):

    raw_surface = surface_from_raw_instance(word_morphemes)
    morph_surface = morph_surface_from_instance(word_morphemes)
    baseline_surface = baseline_surface_from_instance(word_morphemes)

    if raw_surface != morph_surface:
        raw_vs_morph_mismatches.append(
            (idx, word_morphemes, raw_surface, morph_surface)
        )

    if raw_surface != baseline_surface:
        raw_vs_baseline_mismatches.append(
            (idx, word_morphemes, raw_surface, baseline_surface)
        )

    if morph_surface != baseline_surface:
        morph_vs_baseline_mismatches.append(
            (idx, word_morphemes, morph_surface, baseline_surface)
        )

print("Direct instance-level validation")
print("=" * 60)

print(
    "Raw → MorphBPE mismatches:",
    len(raw_vs_morph_mismatches)
)

print(
    "Raw → Baseline mismatches:",
    len(raw_vs_baseline_mismatches)
)

print(
    "MorphBPE → Baseline mismatches:",
    len(morph_vs_baseline_mismatches)
)

```

```

Direct instance-level validation
=====
Raw → MorphBPE mismatches: 0

```

Raw → Baseline mismatches: 0
MorphBPE → Baseline mismatches: 0

```
# =====
# CELL 11: SHOW FIRST DIRECT MISMATCH
# =====

if morph_vs_baseline_mismatches:

    idx, raw_entry, morph_surface, baseline_surface = (
        morph_vs_baseline_mismatches[0]
    )

    print("First MorphBPE/Baseline mismatch")
    print("=" * 60)

    print("Index:", idx)
    print("\nRaw entry:")
    print(raw_entry)

    print("\nRaw surface:")
    print(repr(surface_from_raw_instance(raw_entry)))

    print("\nMorphBPE surface:")
    print(repr(morph_surface))

    print("\nBaseline surface:")
    print(repr(baseline_surface))

else:
    print("No direct MorphBPE/Baseline mismatches found.")
```

No direct MorphBPE/Baseline mismatches found.

```
# =====
# CELL 12: FINAL SURFACE EQUIVALENCE CHECK
# =====

raw_surface_counts = defaultdict(int)
morph_surface_counts_final = defaultdict(int)
baseline_surface_counts_final = defaultdict(int)

for word_morphemes in train_raw:

    # Authoritative surface
    raw_surface = surface_from_raw_instance(word_morphemes)
    raw_surface_counts[raw_surface] += 1

    # MorphBPE representation
    morph_surface = morph_surface_from_instance(word_morphemes)
    morph_surface_counts_final[morph_surface] += 1

    # Baseline representation
    baseline_surface = baseline_surface_from_instance(word_morphemes)
    baseline_surface_counts_final[baseline_surface] += 1

print("FINAL SURFACE-EQUIVALENCE VALIDATION")
print("=" * 60)

print(f"Raw instances:      {sum(raw_surface_counts.values()):,}")
print(f"MorphBPE instances: {sum(morph_surface_counts_final.values()):,}")
print(f"Baseline instances: {sum(baseline_surface_counts_final.values()):,}")

print()
print(f"Raw unique surfaces: {len(raw_surface_counts):,}")
print(f"MorphBPE unique surfaces: {len(morph_surface_counts_final):,}")
print(f"Baseline unique surfaces: {len(baseline_surface_counts_final):,}")

print()
print(
    "Raw == MorphBPE:",
    raw_surface_counts == morph_surface_counts_final
)

print(
    "Raw == Baseline:",
    raw_surface_counts == baseline_surface_counts_final
)

print(
    "MorphBPE == Baseline:",
    morph_surface_counts_final == baseline_surface_counts_final
)
```

FINAL SURFACE-EQUIVALENCE VALIDATION

```
Raw instances:      750,458
MorphBPE instances: 750,458
Baseline instances: 750,458
```

```
Raw unique surfaces: 42,497
MorphBPE unique surfaces: 42,497
Baseline unique surfaces: 42,497
```

```
Raw == MorphBPE: True
Raw == Baseline: True
MorphBPE == Baseline: True
```

```
# =====
# CELL 13: MORPHBPE TRAINER
# =====
#
# Purpose:
#   Train BPE with morphological boundaries treated as
#   hard constraints.
#
# Input:
#   morph_corpus
#
# Representation:
#   WordTuple = (
#       morpheme_1,
#       morpheme_2,
#       ...
#   )
#
#   Each morpheme is a tuple of atomic units:
#
#       (("a",), ("g", "a"))
#
#   means:
#
#       a | ga
#
# Critical constraint:
#   Pair extraction occurs ONLY within individual morphemes.
#   Therefore, BPE can never learn a merge crossing a
#   supplied morphological boundary.
# =====

from collections import defaultdict
from typing import Optional, List, Tuple, Dict

WordTuple = Tuple[Tuple[str, ...], ...]

class MorphBPETrainer:
    def __init__(
        self,
        special_tokens: Optional[List[str]] = None
    ):
        self.special_tokens = special_tokens or [
            "<unk>",
            "<s>",
            "</s>",
            "<pad>",
            "<mask morph>"
        ]

        self.merges: List[Tuple[str, str]] = []
        self.vocab: Dict[str, int] = {}

        # -----
        # Extract pairs WITHIN morphemes only
        # -----

    def _extract_word_pairs(
        self,
        word_tuple: WordTuple
    ) -> List[Tuple[str, str]]:

        pairs = []

        for morpheme in word_tuple:

            for i in range(len(morpheme) - 1):

                pairs.append(
```

```

        (
            morpheme[i],
            morpheme[i + 1]
        )
    )

    return pairs

# -----
# Apply one merge WITHIN each morpheme
# -----

def _apply_merge_to_morpheme(
    self,
    morpheme: Tuple[str, ...],
    pair: Tuple[str, str]
) -> Tuple[str, ...]:

    first, second = pair
    merged = first + second

    output = []

    i = 0

    while i < len(morpheme):

        if (
            i < len(morpheme) - 1
            and morpheme[i] == first
            and morpheme[i + 1] == second
        ):

            output.append(merged)
            i += 2

        else:

            output.append(morpheme[i])
            i += 1

    return tuple(output)

# -----
# Training
# -----

def train(
    self,
    corpus: Dict[WordTuple, int],
    target_vocab_size: int
):

    self.merges = []

    # Initial atomic vocabulary
    initial_units = {
        unit
        for word in corpus
        for morpheme in word
        for unit in morpheme
    }

    current_vocab = (
        list(self.special_tokens)
        + sorted(initial_units)
    )

    self.vocab = {
        token: idx
        for idx, token in enumerate(current_vocab)
    }

    # Working copy of frequency corpus
    working_corpus = dict(corpus)

    pair_counts = defaultdict(int)
    where_pair = defaultdict(set)

    # -----
    # Initial pair counts
    # -----

    for word_tuple, frequency in working_corpus.items():

        for pair in self._extract_word_pairs(word_tuple):

```

```

        pair_counts[pair] += frequency
        where_pair[pair].add(word_tuple)

# -----
# BPE learning loop
# -----

while len(self.vocab) < target_vocab_size:

    if not pair_counts:
        break

    best_pair = max(
        pair_counts,
        key=pair_counts.get
    )

    if pair_counts[best_pair] <= 1:
        break

    self.merges.append(best_pair)

    merged_token = (
        best_pair[0] + best_pair[1]
    )

    if merged_token not in self.vocab:

        self.vocab[merged_token] = (
            len(self.vocab)
        )

    words_to_update = list(
        where_pair[best_pair]
    )

    del where_pair[best_pair]
    del pair_counts[best_pair]

# -----
# Update affected words
# -----

for old_word in words_to_update:

    if old_word not in working_corpus:
        continue

    frequency = working_corpus.pop(
        old_word
    )

    # Remove old pair contributions
    old_pairs = self._extract_word_pairs(
        old_word
    )

    for pair in old_pairs:

        if pair == best_pair:
            continue

        pair_counts[pair] -= frequency

        if pair_counts[pair] <= 0:
            pair_counts.pop(pair, None)

        where_pair[pair].discard(
            old_word
        )

    # Apply merge within morphemes
    new_word = tuple(
        self._apply_merge_to_morpheme(
            morpheme,
            best_pair
        )
        for morpheme in old_word
    )

    working_corpus[new_word] = (
        working_corpus.get(new_word, 0)
        + frequency
    )

```

```

        # Add new pair contributions
        new_pairs = self._extract_word_pairs(
            new_word
        )

        for pair in new_pairs:

            pair_counts[pair] += frequency
            where_pair[pair].add(new_word)

    return self

# -----
# Export vocabulary
# -----

def export_vocab(
    self,
    filepath
):

    with open(
        filepath,
        "w",
        encoding="utf-8"
    ) as f:

        json.dump(
            self.vocab,
            f,
            ensure_ascii=False,
            indent=2
        )

# -----
# Export merges
# -----

def export_merges(
    self,
    filepath
):

    with open(
        filepath,
        "w",
        encoding="utf-8"
    ) as f:

        f.write("#version: 0.2\n")

        for first, second in self.merges:

            f.write(
                f"{first} {second}\n"
            )

print("MorphBPE trainer defined.")

```

MorphBPE trainer defined.

```

# =====
# CELL 14: BASELINE BPE TRAINER
# =====
#
# Purpose:
#   Train an unconstrained BPE baseline.
#
# Critical distinction:
#
#   MorphBPE:
#       [morpheme_1] | [morpheme_2]
#           ↑
#       boundary protected
#
#   Baseline:
#       morpheme_1 + morpheme_2
#           ↑
#       boundary removed
#
# Both systems therefore start from the SAME underlying
# reconstructed corpus.
# =====

```

```

class BaselineBPETrainer(MorphBPETrainer):

    def _extract_word_pairs(
        self,
        word_tuple: WordTuple
    ) -> List[Tuple[str, str]]:

        # -----
        # Collapse all morphemes into one surface sequence.
        # -----

        surface_units = tuple(
            unit
            for morpheme in word_tuple
            for unit in morpheme
        )

        pairs = []

        for i in range(
            len(surface_units) - 1
        ):

            pairs.append(
                (
                    surface_units[i],
                    surface_units[i + 1]
                )
            )

        return pairs

    # -----
    # Baseline merge application
    # -----
    #
    # Unlike MorphBPE, the baseline treats the entire word
    # as one sequence. Therefore merges may cross original
    # morphological boundaries.
    # -----

    def _apply_merge_to_morpheme(
        self,
        morpheme: Tuple[str, ...],
        pair: Tuple[str, str]
    ) -> Tuple[str, ...]:

        # This method is not used directly by the overridden
        # training procedure below.
        return super()._apply_merge_to_morpheme(
            morpheme,
            pair
        )

    def train(
        self,
        corpus: Dict[WordTuple, int],
        target_vocab_size: int
    ):

        self.merges = []

        # Initial vocabulary is identical in principle to
        # MorphBPE: same atomic units and same special tokens.

        initial_units = {
            unit
            for word in corpus
            for morpheme in word
            for unit in morpheme
        }

        current_vocab = (
            list(self.special_tokens)
            + sorted(initial_units)
        )

        self.vocab = {
            token: idx
            for idx, token in enumerate(current_vocab)
        }

        # -----
        # Collapse each word to a single sequence.
        # -----

```

```

working_corpus = {

    tuple(
        unit
        for morpheme in word
        for unit in morpheme
    ): frequency

    for word, frequency in corpus.items()
}

pair_counts = defaultdict(int)
where_pair = defaultdict(set)

# -----
# Initial pair statistics
# -----

for word_sequence, frequency in (
    working_corpus.items()
):

    for i in range(
        len(word_sequence) - 1
    ):

        pair = (
            word_sequence[i],
            word_sequence[i + 1]
        )

        pair_counts[pair] += frequency
        where_pair[pair].add(
            word_sequence
        )

# -----
# BPE learning loop
# -----

while len(self.vocab) < target_vocab_size:

    if not pair_counts:
        break

    best_pair = max(
        pair_counts,
        key=pair_counts.get
    )

    if pair_counts[best_pair] <= 1:
        break

    self.merges.append(best_pair)

    merged_token = (
        best_pair[0]
        + best_pair[1]
    )

    if merged_token not in self.vocab:

        self.vocab[merged_token] = (
            len(self.vocab)
        )

    words_to_update = list(
        where_pair[best_pair]
    )

    del where_pair[best_pair]
    del pair_counts[best_pair]

    # -----
    # Update affected words
    # -----

    for old_word in words_to_update:

        if old_word not in working_corpus:
            continue

        frequency = working_corpus.pop(
            old_word
        )

```



```

# Remove old pair contributions
for i in range(
    len(old_word) - 1
):

    pair = (
        old_word[i],
        old_word[i + 1]
    )

    if pair == best_pair:
        continue

    pair_counts[pair] -= frequency

    if pair_counts[pair] <= 0:
        pair_counts.pop(
            pair,
            None
        )

    where_pair[pair].discard(
        old_word
    )

# Apply merge
first, second = best_pair
merged = first + second

new_word = []
i = 0

while i < len(old_word):

    if (
        i < len(old_word) - 1
        and old_word[i] == first
        and old_word[i + 1] == second
    ):

        new_word.append(merged)
        i += 2

    else:

        new_word.append(
            old_word[i]
        )
        i += 1

new_word = tuple(new_word)

working_corpus[new_word] = (
    working_corpus.get(
        new_word,
        0
    )
    + frequency
)

# Add new pair contributions
for i in range(
    len(new_word) - 1
):

    pair = (
        new_word[i],
        new_word[i + 1]
    )

    pair_counts[pair] += frequency
    where_pair[pair].add(
        new_word
    )

return self

```

```
print("Baseline BPE trainer defined.")
```

```
Baseline BPE trainer defined.
```

```

# =====
# CELL 15: TRAIN MORPHBPE MODELS

```

```
# =====

TARGET_VOCAB_SIZES = [
    2000,
    4000,
    8000,
    12000,
    16000
]

morph_models = {}

print("Training MorphBPE models")
print("=" * 70)

for target_vocab_size in TARGET_VOCAB_SIZES:

    print(
        f"\nTraining MorphBPE "
        f"(target vocabulary = {target_vocab_size:,})"
    )

    trainer = MorphBPETrainer()

    trainer.train(
        morph_corpus,
        target_vocab_size=target_vocab_size
    )

    vocab_path = (
        MODEL_DIR
        / f"m_{target_vocab_size}_vocab.json"
    )

    merges_path = (
        MODEL_DIR
        / f"m_{target_vocab_size}_merges.txt"
    )

    trainer.export_vocab(vocab_path)
    trainer.export_merges(merges_path)

    morph_models[target_vocab_size] = {
        "vocab_size": len(trainer.vocab),
        "num_merges": len(trainer.merges),
        "vocab_path": vocab_path,
        "merges_path": merges_path
    }

    print(
        f"Actual vocabulary: "
        f"{len(trainer.vocab):,}"
    )

    print(
        f"Learned merges: "
        f"{len(trainer.merges):,}"
    )

print("\nMorphBPE training complete.")
```

```
Training MorphBPE models
=====

Training MorphBPE (target vocabulary = 2,000)
Actual vocabulary: 2,000
Learned merges:    1,818

Training MorphBPE (target vocabulary = 4,000)
Actual vocabulary: 4,000
Learned merges:    3,818

Training MorphBPE (target vocabulary = 8,000)
Actual vocabulary: 8,000
Learned merges:    7,818

Training MorphBPE (target vocabulary = 12,000)
Actual vocabulary: 12,000
Learned merges:    11,818

Training MorphBPE (target vocabulary = 16,000)
Actual vocabulary: 16,000
Learned merges:    15,818

MorphBPE training complete.
```

```
# =====
# CELL 16: TRAIN BASELINE BPE MODELS
# =====

baseline_models = {}

print("Training Baseline BPE models")
print("=" * 70)

for target_vocab_size in TARGET_VOCAB_SIZES:

    print(
        f"\nTraining Baseline BPE "
        f"(target vocabulary = {target_vocab_size:,})"
    )

    trainer = BaselineBPETrainer()

    trainer.train(
        morph_corpus,
        target_vocab_size=target_vocab_size
    )

    vocab_path = (
        MODEL_DIR
        / f"baseline_{target_vocab_size}_vocab.json"
    )

    merges_path = (
        MODEL_DIR
        / f"baseline_{target_vocab_size}_merges.txt"
    )

    trainer.export_vocab(vocab_path)
    trainer.export_merges(merges_path)

    baseline_models[target_vocab_size] = {
        "vocab_size": len(trainer.vocab),
        "num_merges": len(trainer.merges),
        "vocab_path": vocab_path,
        "merges_path": merges_path
    }

    print(
        f"Actual vocabulary: "
        f"{len(trainer.vocab):,}"
    )

    print(
        f"Learned merges: "
        f"{len(trainer.merges):,}"
    )

print("\nBaseline BPE training complete.")
```

```
Training Baseline BPE models
=====

Training Baseline BPE (target vocabulary = 2,000)
Actual vocabulary: 2,000
Learned merges:    1,818

Training Baseline BPE (target vocabulary = 4,000)
Actual vocabulary: 4,000
Learned merges:    3,818

Training Baseline BPE (target vocabulary = 8,000)
Actual vocabulary: 8,000
Learned merges:    7,818

Training Baseline BPE (target vocabulary = 12,000)
Actual vocabulary: 12,000
Learned merges:    11,818

Training Baseline BPE (target vocabulary = 16,000)
Actual vocabulary: 16,000
Learned merges:    15,818

Baseline BPE training complete.
```

```
# =====
# CELL 17: VALIDATE TRAINED BPE ARTIFACTS
# =====
#
# PURPOSE
# -----
# Validate that all trained BPE artifacts are structurally
```

```

# usable before downstream evaluation.
#
# Validation checks:
# 1. Vocabulary size matches the requested size.
# 2. Merge file contains only valid two-symbol merges.
# 3. No learned merge contains whitespace.
# 4. No learned vocabulary token contains corpus whitespace.
# 5. Intentional special tokens containing spaces are allowed.
#
# IMPORTANT
# -----
# <mask morph> contains a space intentionally and therefore
# MUST NOT be treated as an invalid whitespace vocabulary item.
# =====

ALLOWED_SPECIAL_TOKENS = {
    "<unk>",
    "<s>",
    "</s>",
    "<pad>",
    "<mask morph>"
}

def validate_bpe_artifact(
    model_name: str,
    model_info: Dict
) -> Dict:

    print("=" * 70)
    print(f"VALIDATING: {model_name}")
    print("=" * 70)

    vocab_path = model_info["vocab_path"]
    merges_path = model_info["merges_path"]

    # -----
    # 1. Load vocabulary
    # -----

    with open(
        vocab_path,
        "r",
        encoding="utf-8"
    ) as f:
        vocab = json.load(f)

    # -----
    # 2. Read and validate merges
    # -----

    merges = []
    invalid_merge_lines = []
    whitespace_merges = []

    with open(
        merges_path,
        "r",
        encoding="utf-8"
    ) as f:

        for line_number, raw_line in enumerate(
            f,
            start=1
        ):

            raw = raw_line.rstrip("\n\r")

            # Header is allowed
            if raw.startswith("#version:"):
                continue

            # Ignore genuinely empty lines
            if not raw.strip():
                continue

            parts = raw.split()

            if len(parts) != 2:

                invalid_merge_lines.append(
                    (
                        line_number,
                        repr(raw)
                    )
                )

```

```

        continue

    first, second = parts

    merges.append(
        (first, second)
    )

    # Learned BPE merges must not contain whitespace
    if (
        any(ch.isspace() for ch in first)
        or any(ch.isspace() for ch in second)
    ):
        whitespace_merges.append(
            (
                line_number,
                first,
                second
            )
        )

# -----
# 3. Identify whitespace vocabulary entries
# -----

whitespace_vocab = [
    token
    for token in vocab
    if any(ch.isspace() for ch in token)
]

# Only intentional special tokens are allowed
forbidden_whitespace_vocab = [
    token
    for token in whitespace_vocab
    if token not in ALLOWED_SPECIAL_TOKENS
]

# -----
# 4. Print validation summary
# -----

print(
    f"Vocabulary entries : {len(vocab):,}"
)

print(
    f"Merge entries      : {len(merges):,}"
)

print(
    f"Invalid merge lines: {len(invalid_merge_lines)}"
)

print(
    f"Whitespace merges  : {len(whitespace_merges)}"
)

print(
    f"Whitespace vocab    : {len(whitespace_vocab)}"
)

if whitespace_vocab:
    print(
        "\nWhitespace vocabulary entries:"
    )

    for token in whitespace_vocab[:20]:
        status = (
            "ALLOWED SPECIAL TOKEN"
            if token in ALLOWED_SPECIAL_TOKENS
            else "FORBIDDEN"
        )

        print(
            f" {repr(token)} -> {status}"
        )

# -----
# 5. Assertions
# -----

assert len(invalid_merge_lines) == 0, (

```

```

        f"{model_name}: invalid merge lines detected: "
        f"{invalid_merge_lines[:10]}"
    )

    assert len(whitespace_merges) == 0, (
        f"{model_name}: learned whitespace merges detected: "
        f"{whitespace_merges[:10]}"
    )

    assert len(forbidden_whitespace_vocab) == 0, (
        f"{model_name}: forbidden whitespace vocabulary "
        f"entries detected: "
        f"{forbidden_whitespace_vocab[:10]}"
    )

# -----
# 6. Check expected vocabulary size
# -----

expected_vocab_size = model_info["vocab_size"]

assert len(vocab) == expected_vocab_size, (
    f"{model_name}: vocabulary size mismatch. "
    f"Expected {expected_vocab_size:},, "
    f"found {len(vocab):},."
)

# -----
# 7. Check merge count
# -----

expected_merge_count = model_info["num_merges"]

assert len(merges) == expected_merge_count, (
    f"{model_name}: merge count mismatch. "
    f"Expected {expected_merge_count:},, "
    f"found {len(merges):},."
)

# -----
# 8. PASS
# -----

result = {
    "model": model_name,
    "vocab_size": len(vocab),
    "merge_count": len(merges),
    "invalid_merge_lines": len(
        invalid_merge_lines
    ),
    "whitespace_merges": len(
        whitespace_merges
    ),
    "whitespace_vocab": len(
        whitespace_vocab
    ),
    "forbidden_whitespace_vocab": len(
        forbidden_whitespace_vocab
    ),
    "status": "PASS"
}

print()
print(
    f"STATUS: PASS - {model_name} artifact "
    f"is structurally valid."
)

return result

# -----
# Validate all MorphBPE models
# -----

morph_validation = {}

for vocab_size in TARGET_VOCAB_SIZES:

    morph_validation[vocab_size] = (
        validate_bpe_artifact(
            f"MorphBPE-{vocab_size // 1000}k",
            morph_models[vocab_size]
        )
    )

```

```
# -----
# Validate all Baseline BPE models
# -----

baseline_validation = {}

for vocab_size in TARGET_VOCAB_SIZES:

    baseline_validation[vocab_size] = (
        validate_bpe_artifact(
            f"BaselineBPE-{vocab_size // 1000}k",
            baseline_models[vocab_size]
        )
    )

print()
print("=" * 70)
print("CELL 17 COMPLETE")
print("=" * 70)
```

```
STATUS: PASS – BaselineBPE-2k artifact is structurally valid.
=====
VALIDATING: BaselineBPE-4k
=====
Vocabulary entries : 4,000
Merge entries      : 3,818
Invalid merge lines: 0
Whitespace merges  : 0
Whitespace vocab    : 1

Whitespace vocabulary entries:
'<mask morph>' -> ALLOWED SPECIAL TOKEN

STATUS: PASS – BaselineBPE-4k artifact is structurally valid.
=====
VALIDATING: BaselineBPE-8k
=====
Vocabulary entries : 8,000
Merge entries      : 7,818
Invalid merge lines: 0
Whitespace merges  : 0
Whitespace vocab    : 1

Whitespace vocabulary entries:
'<mask morph>' -> ALLOWED SPECIAL TOKEN

STATUS: PASS – BaselineBPE-8k artifact is structurally valid.
=====
VALIDATING: BaselineBPE-12k
=====
Vocabulary entries : 12,000
Merge entries      : 11,818
Invalid merge lines: 0
Whitespace merges  : 0
Whitespace vocab    : 1

Whitespace vocabulary entries:
'<mask morph>' -> ALLOWED SPECIAL TOKEN

STATUS: PASS – BaselineBPE-12k artifact is structurally valid.
=====
VALIDATING: BaselineBPE-16k
=====
Vocabulary entries : 16,000
Merge entries      : 15,818
Invalid merge lines: 0
Whitespace merges  : 0
Whitespace vocab    : 1

Whitespace vocabulary entries:
'<mask morph>' -> ALLOWED SPECIAL TOKEN

STATUS: PASS – BaselineBPE-16k artifact is structurally valid.

=====
CELL 17 COMPLETE
=====
```

```
# =====
# CELL 18: FREEZE DOWNSTREAM EXPERIMENTAL CONFIGURATION
# =====
#
# Purpose:
# Freeze the exact tokenizer/model conditions that will be
# used for downstream evaluation.
#
# Selected tokenizer:
```

```

# MorphBPE-8k
# Baseline-8k
#
# Experimental principle:
# The tokenizer is the intended independent variable.
# All other experimental conditions should remain identical.
# =====

SELECTED_VOCAB_SIZE = 8000

DOWNSTREAM_CONFIG = {
    "selected_vocab_size": SELECTED_VOCAB_SIZE,

    "tokenizers": [
        "MorphBPE",
        "Baseline"
    ],

    "model_families": [
        "RoBERTa-Tiny",
        "Llama-Tiny"
    ],

    "same_training_corpus": True,
    "same_data_split": True,
    "same_vocab_size": True,
    "same_model_configuration": True,
    "same_training_budget": True,
    "same_evaluation_data": True,

    # Reproducibility
    "random_seed": 42
}

# -----
# Validate selected artifacts
# -----

assert SELECTED_VOCAB_SIZE in morph_models
assert SELECTED_VOCAB_SIZE in baseline_models

assert (
    morph_models[SELECTED_VOCAB_SIZE]["vocab_size"]
    == SELECTED_VOCAB_SIZE
)

assert (
    baseline_models[SELECTED_VOCAB_SIZE]["vocab_size"]
    == SELECTED_VOCAB_SIZE
)

# -----
# Print frozen configuration
# -----

print("=" * 70)
print("DOWNSTREAM EXPERIMENTAL CONFIGURATION")
print("=" * 70)

print(
    f"Selected vocabulary size : "
    f"{SELECTED_VOCAB_SIZE:,"}
)

print("Tokenizer comparison      : MorphBPE vs Baseline")
print("Encoder model              : RoBERTa-Tiny")
print("Decoder model               : Llama-Tiny")
print("Random seed                 : 42")

print("\nControlled conditions:")
print(" Same training corpus      :", DOWNSTREAM_CONFIG["same_training_corpus"])
print(" Same data split           :", DOWNSTREAM_CONFIG["same_data_split"])
print(" Same vocabulary size      :", DOWNSTREAM_CONFIG["same_vocab_size"])
print(" Same model config         :", DOWNSTREAM_CONFIG["same_model_configuration"])
print(" Same training budget      :", DOWNSTREAM_CONFIG["same_training_budget"])
print(" Same evaluation data      :", DOWNSTREAM_CONFIG["same_evaluation_data"])

print("\nMorphBPE-8k:")
print(" Vocab :", morph_models[8000]["vocab_path"])
print(" Merges:", morph_models[8000]["merges_path"])

print("\nBaseline-8k:")
print(" Vocab :", baseline_models[8000]["vocab_path"])
print(" Merges:", baseline_models[8000]["merges_path"])

```



```
print("\nSTATUS: PASS – downstream configuration frozen.")
```

```
=====
DOWNSTREAM EXPERIMENTAL CONFIGURATION
=====
Selected vocabulary size : 8,000
Tokenizer comparison      : MorphBPE vs Baseline
Encoder model            : RoBERTa-Tiny
Decoder model            : Llama-Tiny
Random seed              : 42

Controlled conditions:
  Same training corpus   : True
  Same data split        : True
  Same vocabulary size   : True
  Same model config      : True
  Same training budget   : True
  Same evaluation data   : True

MorphBPE-8k:
  Vocab : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/m_8000_vocab.json
  Merges: /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/m_8000_merges.txt

Baseline-8k:
  Vocab : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/baseline_8000_vocab.json
  Merges: /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/baseline_8000_merges.txt

STATUS: PASS – downstream configuration frozen.
```

```
# =====
# CELL 19: PREPARE DOWNSTREAM TEXT DATA
# =====
#
# Purpose:
#   Reconstruct surface-form text from the same corpus used
#   to train the tokenizers.
#
# Important:
#   No new corpus is introduced.
#   The downstream data originate from the same train/eval
#   instances used in the tokenizer experiment.
# =====

def reconstruct_surface_word(word_tuple):
    """
    Reconstruct a word from its morphological representation.
    """

    return "".join(
        "".join(morpheme)
        for morpheme in word_tuple
    )

def reconstruct_surface_corpus(corpus):
    """
    Convert a frequency dictionary of WordTuple instances
    into a list of surface-form word instances.
    """

    surface_words = []

    for word_tuple, frequency in corpus.items():

        surface_word = reconstruct_surface_word(
            word_tuple
        )

        for _ in range(frequency):
            surface_words.append(
                surface_word
            )

    return surface_words

# -----
# Build downstream training data
# -----

downstream_train_words = reconstruct_surface_corpus(
    morph_corpus
)

# -----
# Build evaluation words from the held-out sample
```

```

# -----

downstream_eval_words = [
    reconstruct_surface_word(word)
    for word in eval_test_sample
]

print("=" * 70)
print("DOWNSTREAM DATA PREPARATION")
print("=" * 70)

print(
    f"Training surface-word instances : "
    f"{len(downstream_train_words):,}"
)

print(
    f"Evaluation surface-word instances: "
    f"{len(downstream_eval_words):,}"
)

# -----
# Basic validation
# -----

assert len(downstream_train_words) > 0
assert len(downstream_eval_words) > 0

# No whitespace-only training instances
whitespace_only = [
    word
    for word in downstream_train_words
    if word.strip() == ""
]

assert not whitespace_only, (
    "Whitespace-only training instances detected."
)

print("\nSTATUS: PASS")

```

```

=====
DOWNSTREAM DATA PREPARATION
=====
Training surface-word instances : 750,457
Evaluation surface-word instances: 10,000

STATUS: PASS

```

```

# =====
# CELL 20: VALIDATE DOWNSTREAM DATA INTEGRITY
# =====
#
# Purpose:
#   Verify that surface reconstruction is lossless relative
#   to the cleaned MorphBPE corpus.
#
# Checks:
#   1. Instance count
#   2. Surface reconstruction
#   3. No unintended whitespace units
#   4. Sample-level consistency
# =====

# -----
# Check number of frequency-expanded instances
# -----

expected_train_instances = sum(
    morph_corpus.values()
)

assert len(downstream_train_words) == (
    expected_train_instances
)

print("=" * 70)
print("DOWNSTREAM DATA INTEGRITY")
print("=" * 70)

print(
    f"Expected training instances : "
    f"{expected_train_instances:,}"
)

```

```

print(
    f"Actual training instances : "
    f"{len(downstream_train_words):,}"
)

# -----
# Validate reconstructed surfaces
# -----

sample_check_count = min(
    100,
    len(downstream_train_words)
)

for word in downstream_train_words[
    :sample_check_count
]:

    assert isinstance(word, str)
    assert len(word) > 0

# -----
# Validate evaluation surfaces
# -----

for word in downstream_eval_words[:100]:

    assert isinstance(word, str)
    assert len(word) > 0

# -----
# Display examples
# -----

print("\nExample reconstructed training words:")

for word in downstream_train_words[:10]:
    print(f" {repr(word)}")

print("\nExample evaluation words:")

for word in downstream_eval_words[:10]:
    print(f" {repr(word)}")

print("\nSTATUS: PASS – downstream data integrity verified.")

```

```

=====
DOWNSTREAM DATA INTEGRITY
=====
Expected training instances : 750,457
Actual training instances   : 750,457

Example reconstructed training words:
'n'
'n'
'n'
'n'
'n'
'n'
'n'
'n'
'n'
'n'

Example evaluation words:
'si'
'ihu'
'ahụ , '
'Boko'
'umụ'
'ga-enye'
'ndị'
'n'
'gbuo'
'gini'

STATUS: PASS – downstream data integrity verified.

```

```

# =====
# CELL 21: CONVERT SELECTED 8K TOKENIZERS TO HUGGING FACE
# =====
#

```

```

# Purpose:
#   Convert the validated 8k MorphBPE and Baseline BPE
#   vocab/merge artifacts into standard Hugging Face
#   `tokenizer.json` files.
#
# These tokenizer.json files are required for:
#   - Hugging Face Tokenizers
#   - Transformers
#   - RoBERTa-style downstream experiments
#   - reproducible tokenizer loading
#
# Important:
#   <mask morph> is an intentional special token and is
#   registered separately rather than treated as a BPE merge.
# =====

from pathlib import Path
import json

from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.pre_tokenizers import Whitespace

# -----
# Paths
# -----

DOWNSTREAM_TOKENIZER_DIR = MODEL_DIR / "downstream_tokenizers"

DOWNSTREAM_TOKENIZER_DIR.mkdir(
    parents=True,
    exist_ok=True
)

# -----
# Special tokens
# -----

SPECIAL_TOKENS = [
    "<unk>",
    "<s>",
    "</s>",
    "<pad>",
    "<mask morph>"
]

# -----
# Conversion function
# -----

def build_hf_tokenizer(
    vocab_path,
    merges_path,
    output_path
):
    vocab_path = Path(vocab_path)
    merges_path = Path(merges_path)
    output_path = Path(output_path)

    # -----
    # Load vocabulary
    # -----

    with open(
        vocab_path,
        "r",
        encoding="utf-8"
    ) as f:
        vocab = json.load(f)

    # -----
    # Load merges
    # -----

    merges = []

    with open(
        merges_path,
        "r",
        encoding="utf-8"
    ) as f:

```

```

    for line_number, raw_line in enumerate(
        f,
        start=1
    ):

        line = raw_line.rstrip("\r\n")

        if not line:
            continue

        if line.startswith("#version:"):
            continue

        parts = line.split()

        if len(parts) != 2:
            raise ValueError(
                f"Invalid merge at line "
                f"{line_number}: {repr(line)}"
            )

        merges.append(
            (parts[0], parts[1])
        )

# -----
# Construct BPE model
# -----

bpe_model = BPE(
    vocab=vocab,
    merges=merges,
    unk_token="<unk>"
)

tokenizer = Tokenizer(
    bpe_model
)

# -----
# Pre-tokenization
#
# We use whitespace splitting for ordinary downstream
# text. Morphological boundaries are NOT supplied here.
#
# This is intentional:
# the learned tokenizer must operate on ordinary surface
# text during inference.
# -----

tokenizer.pre_tokenizer = Whitespace()

# -----
# Register special tokens
# -----

tokenizer.add_special_tokens(
    [
        token
        for token in SPECIAL_TOKENS
        if token in vocab
    ]
)

# -----
# Save tokenizer.json
# -----

tokenizer.save(
    str(output_path)
)

return tokenizer

# =====
# MorphBPE-8k
# =====

morph_8k_tokenizer_json = (
    DOWNSTREAM_TOKENIZER_DIR
    / "morphbpe_8k_tokenizer.json"
)

morph_8k_hf_tokenizer = build_hf_tokenizer(

```

```

        morph_models[8000]["vocab_path"],
        morph_models[8000]["merges_path"],
        morph_8k_tokenizer_json
    )

# =====
# Baseline BPE-8k
# =====

baseline_8k_tokenizer_json = (
    DOWNSTREAM_TOKENIZER_DIR
    / "baselinebpe_8k_tokenizer.json"
)

baseline_8k_hf_tokenizer = build_hf_tokenizer(
    baseline_models[8000]["vocab_path"],
    baseline_models[8000]["merges_path"],
    baseline_8k_tokenizer_json
)

# =====
# Report
# =====

print("=" * 70)
print("HUGGING FACE TOKENIZER CONVERSION")
print("=" * 70)

print(
    "MorphBPE-8k tokenizer:"
)
print(
    f"    {morph_8k_tokenizer_json}"
)

print(
    "\nBaselineBPE-8k tokenizer:"
)
print(
    f"    {baseline_8k_tokenizer_json}"
)

print("\nSTATUS: PASS – tokenizer.json files created.")

```

```

=====
HUGGING FACE TOKENIZER CONVERSION
=====
MorphBPE-8k tokenizer:
    /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/downstream_tokenizers/morphbpe_8k_tokenizer.json

BaselineBPE-8k tokenizer:
    /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/downstream_tokenizers/baselinebpe_8k_tokenizer.json

STATUS: PASS – tokenizer.json files created.

```

```

# =====
# CELL 22: VERIFY STANDARD HUGGING FACE LOADING
# =====
#
# Purpose:
#   Confirm that the generated tokenizer.json files can be
#   loaded independently using:
#
#       Tokenizer.from_file(...)
#
# This verifies that the downstream artifacts are portable
# and compatible with the standard Hugging Face Tokenizers
# library.
# =====

from tokenizers import Tokenizer

# -----
# Reload MorphBPE
# -----

morph_8k_loaded = Tokenizer.from_file(
    str(morph_8k_tokenizer_json)
)

# -----
# Reload Baseline BPE

```

```
# -----

baseline_8k_loaded = Tokenizer.from_file(
    str(baseline_8k_tokenizer_json)
)

# -----
# Basic checks
# -----

assert morph_8k_loaded.get_vocab_size() == 8000

assert baseline_8k_loaded.get_vocab_size() == 8000

print("=" * 70)
print("HUGGING FACE TOKENIZER RELOAD TEST")
print("=" * 70)

print(
    "MorphBPE-8k vocabulary:",
    morph_8k_loaded.get_vocab_size()
)

print(
    "BaselineBPE-8k vocabulary:",
    baseline_8k_loaded.get_vocab_size()
)

print(
    "\nMorphBPE tokenizer.json:",
    "LOAD SUCCESS"
)

print(
    "Baseline tokenizer.json:",
    "LOAD SUCCESS"
)

print(
    "\nSTATUS: PASS – both tokenizers load with "
    "Tokenizer.from_file(...)."
)
```

```
=====
HUGGING FACE TOKENIZER RELOAD TEST
=====
MorphBPE-8k vocabulary: 8000
BaselineBPE-8k vocabulary: 8000

MorphBPE tokenizer.json: LOAD SUCCESS
Baseline tokenizer.json: LOAD SUCCESS

STATUS: PASS – both tokenizers load with Tokenizer.from_file(...).
```

```
# =====
# CELL 23: SURFACE TOKENIZATION SANITY CHECK
# =====
#
# Purpose:
#   Compare MorphBPE-8k and BaselineBPE-8k on identical
#   surface-form inputs.
#
# No morphological annotations are supplied at inference.
# =====

comparison_words = [
    "kwuo",
    "chɔrɔ",
    "amalitela",
    "buru",
    "ihazighari",
    "ikpachara",
    "naijiria",
    "ɔfala",
    "mmemme",
    "akwukwo"
]

print("=" * 70)
print("SURFACE TOKENIZATION COMPARISON")
print("=" * 70)

for word in comparison_words:
```

```

morph_encoding = morph_8k_loaded.encode(
    word
)

baseline_encoding = baseline_8k_loaded.encode(
    word
)

print("\nWord:", repr(word))

print(
    "  MorphBPE :",
    morph_encoding.tokens
)

print(
    "  Baseline :",
    baseline_encoding.tokens
)

```

===== SURFACE TOKENIZATION COMPARISON =====

```

Word: 'kwuo'
MorphBPE : ['k', 'wu', 'o']
Baseline : ['k', 'wuo']

Word: 'chɔrɔ'
MorphBPE : ['c', 'hɔ', 'rɔ']
Baseline : ['c', 'hɔrɔ']

Word: 'amalitela'
MorphBPE : ['ama', 'lite', 'la']
Baseline : ['amalite', 'la']

Word: 'bɔrɔ'
MorphBPE : ['bɔ', 'rɔ']
Baseline : ['bɔrɔ']

Word: 'ɪhazighari'
MorphBPE : ['ɪ', 'ha', 'zig', 'ha', 'ri']
Baseline : ['ɪhazi', 'g', 'h', 'ari']

Word: 'ɪkpachara'
MorphBPE : ['ɪ', 'k', 'pa', 'c', 'hara']
Baseline : ['ɪ', 'k', 'pa', 'c', 'hara']

Word: 'naijirɪa'
MorphBPE : ['naijirɪa']
Baseline : ['naijirɪa']

Word: 'ɔfala'
MorphBPE : ['ɔfala']
Baseline : ['ɔf', 'ala']

Word: 'mmemme'
MorphBPE : ['mmemme']
Baseline : ['mmemme']

Word: 'akwɪkwɔ'
MorphBPE : ['a', 'k', 'wɪ', 'k', 'wɔ']
Baseline : ['ak', 'wɪ', 'k', 'wɔ']

```

```

# =====
# CELL 24: TOKENIZATION RECONSTRUCTION CHECK
# =====
#
# Purpose:
#   Verify that both tokenizers can encode the selected
#   surface words without producing an unknown token.
# =====

test_words = downstream_eval_words[:1000]

morph_unk_id = morph_8k_loaded.token_to_id("<unk>")
baseline_unk_id = baseline_8k_loaded.token_to_id("<unk>")

morph_unknowns = 0
baseline_unknowns = 0

for word in test_words:

    morph_encoding = morph_8k_loaded.encode(
        word
    )

```



```

)

baseline_encoding = baseline_8k_loaded.encode(
    word
)

if morph_unk_id in morph_encoding.ids:
    morph_unknowns += 1

if baseline_unk_id in baseline_encoding.ids:
    baseline_unknowns += 1

print("=" * 70)
print("TOKENIZATION COVERAGE CHECK")
print("=" * 70)

print(
    f"Evaluation words checked : {len(test_words):,}"
)

print(
    f"MorphBPE <unk> cases      : {morph_unknowns:,}"
)

print(
    f"Baseline <unk> cases      : {baseline_unknowns:,}"
)

```

```

=====
TOKENIZATION COVERAGE CHECK
=====
Evaluation words checked : 1,000
MorphBPE <unk> cases      : 0
Baseline <unk> cases      : 0

```

```

# =====
# CELL 22 – CONTROLLED ROBERTA-TINY ARCHITECTURE
# =====
# Purpose:
#   Construct two architecturally identical RoBERTa-Tiny models around the
#   selected 8k vocabulary.
#
# Experimental principle:
#   The ONLY intended difference between the two systems is the tokenizer:
#
#       MorphBPE-8k → RoBERTa-Tiny
#       BaselineBPE-8k → RoBERTa-Tiny
#
# Everything else is held constant.
# =====

import os
import random
import numpy as np
import torch

from transformers import RobertaConfig, RobertaForMaskedLM

# -----
# 1. Reproducibility
# -----

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed(SEED)
    torch.cuda.manual_seed_all(SEED)

# Deterministic behavior where possible
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

print("=" * 80)
print("CONTROLLED ROBERTA-TINY SETUP")
print("=" * 80)
print(f"Random seed: {SEED}")
print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")

if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")

```

```

# -----
# 2. Selected vocabulary size
# -----

VOCAB_SIZE = 8_000

print("\nSelected vocabulary size:", VOCAB_SIZE)

# -----
# 3. Define ONE architecture configuration
# -----
# This configuration is shared by BOTH experiments.
#
# We deliberately instantiate from configuration rather than loading an
# existing pretrained RoBERTa checkpoint.
#
# This means:
# - same number of layers
# - same hidden size
# - same attention heads
# - same intermediate size
# - same positional capacity
# - same vocabulary size
# -----

ROBERTA_TINY_CONFIG = RobertaConfig(
    vocab_size=VOCAB_SIZE,

    # RoBERTa special-token IDs
    bos_token_id=0,
    eos_token_id=2,
    pad_token_id=3,

    # Tiny encoder architecture
    hidden_size=256,
    num_hidden_layers=4,
    num_attention_heads=4,
    intermediate_size=1024,

    # Sequence length
    max_position_embeddings=512,

    # RoBERTa-style settings
    hidden_act="gelu",
    hidden_dropout_prob=0.1,
    attention_probs_dropout_prob=0.1,

    # Layer normalization
    layer_norm_eps=1e-5,

    # MLM
    type_vocab_size=1,

    # Keep output vocabulary tied to input embeddings
    tie_word_embeddings=True,
)

print("\nArchitecture configuration:")
print("-" * 80)

for key, value in ROBERTA_TINY_CONFIG.to_dict().items():
    if key in [
        "vocab_size",
        "hidden_size",
        "num_hidden_layers",
        "num_attention_heads",
        "intermediate_size",
        "max_position_embeddings",
        "hidden_dropout_prob",
        "attention_probs_dropout_prob",
        "tie_word_embeddings",
    ]:
        print(f"{key:30s}: {value}")

# -----
# 4. Construct MorphBPE model
# -----

morph_roberta = RobertaForMaskedLM(
    ROBERTA_TINY_CONFIG
)

```

```

# -----
# 5. Construct Baseline BPE model
# -----

# Recreate the identical configuration rather than modifying the MorphBPE
# model. This guarantees that both models start from the same architecture.

baseline_config = RobertaConfig.from_dict(
    ROBERTA_TINY_CONFIG.to_dict()
)

baseline_roberta = RobertaForMaskedLM(
    baseline_config
)

# -----
# 6. Parameter-count verification
# -----

def count_parameters(model):
    total = sum(p.numel() for p in model.parameters())
    trainable = sum(
        p.numel() for p in model.parameters()
        if p.requires_grad
    )
    return total, trainable

morph_total, morph_trainable = count_parameters(morph_roberta)
baseline_total, baseline_trainable = count_parameters(baseline_roberta)

print("\n" + "=" * 80)
print("PARAMETER BUDGET VERIFICATION")
print("=" * 80)

print(f"MorphBPE total parameters      : {morph_total:,}")
print(f"Baseline total parameters        : {baseline_total:,}")

print(f"MorphBPE trainable parameters: {morph_trainable:,}")
print(f"Baseline trainable parameters: {baseline_trainable:,}")

# -----
# 7. Hard equality checks
# -----

assert morph_roberta.config.vocab_size == VOCAB_SIZE
assert baseline_roberta.config.vocab_size == VOCAB_SIZE

assert morph_total == baseline_total
assert morph_trainable == baseline_trainable

assert morph_roberta.config.hidden_size == baseline_roberta.config.hidden_size
assert (
    morph_roberta.config.num_hidden_layers
    == baseline_roberta.config.num_hidden_layers
)
assert (
    morph_roberta.config.num_attention_heads
    == baseline_roberta.config.num_attention_heads
)
assert (
    morph_roberta.config.intermediate_size
    == baseline_roberta.config.intermediate_size
)
assert (
    morph_roberta.config.max_position_embeddings
    == baseline_roberta.config.max_position_embeddings
)

print("\nARCHITECTURE CHECK: PASS")
print("Both models have:")
print(f" Vocabulary size : {VOCAB_SIZE:,}")
print(f" Parameters      : {morph_total:,}")
print(f" Hidden size     :", morph_roberta.config.hidden_size)
print(f" Layers          :", morph_roberta.config.num_hidden_layers)
print(f" Attention heads:", morph_roberta.config.num_attention_heads)
print(f" Intermediate    :", morph_roberta.config.intermediate_size)
print(f" Max sequence    :", morph_roberta.config.max_position_embeddings)

print("\n" + "=" * 80)

```

```
print("CONTROLLED MODEL PAIR READY")
print("=" * 80)
```

```
=====
CONTROLLED ROBERTA-TINY SETUP
=====
```

```
Random seed: 42
PyTorch version: 2.11.0+cu128
CUDA available: True
GPU: Tesla T4
```

```
Selected vocabulary size: 8000
```

```
Architecture configuration:
```

```
-----
vocab_size           : 8000
hidden_size          : 256
num_hidden_layers    : 4
num_attention_heads   : 4
intermediate_size     : 1024
hidden_dropout_prob   : 0.1
attention_probs_dropout_prob : 0.1
max_position_embeddings : 512
tie_word_embeddings   : True
```

```
=====
PARAMETER BUDGET VERIFICATION
=====
```

```
MorphBPE total parameters : 5,413,184
Baseline total parameters : 5,413,184
MorphBPE trainable parameters: 5,413,184
Baseline trainable parameters: 5,413,184
```

```
ARCHITECTURE CHECK: PASS
```

```
Both models have:
```

```
  Vocabulary size : 8,000
  Parameters       : 5,413,184
  Hidden size      : 256
  Layers           : 4
  Attention heads  : 4
  Intermediate     : 1024
  Max sequence     : 512
```

```
=====
CONTROLLED MODEL PAIR READY
=====
```

```
# =====
# CELL 23 - IDENTICAL INITIAL WEIGHTS
# =====

# Reinitialize both models from the exact same random state.

torch.manual_seed(SEED)

morph_roberta = RobertaForMaskedLM(
    RobertaConfig.from_dict(ROBERTA_TINY_CONFIG.to_dict())
)

# Capture the exact MorphBPE initial state
morph_initial_state = {
    name: param.detach().clone()
    for name, param in morph_roberta.state_dict().items()
}

# Give BaselineBPE the exact same initial parameters
baseline_roberta = RobertaForMaskedLM(
    RobertaConfig.from_dict(ROBERTA_TINY_CONFIG.to_dict())
)

baseline_roberta.load_state_dict(morph_initial_state)

# -----
# Verify exact equality
# -----

for name, morph_param in morph_roberta.state_dict().items():
    baseline_param = baseline_roberta.state_dict()[name]

    assert torch.equal(
        morph_param,
        baseline_param
    ), f"Initial parameter mismatch: {name}"

print("=" * 80)
print("INITIAL WEIGHT CONTROL")
print("=" * 80)
```

```
print("MorphBPE and BaselineBPE models have EXACTLY identical initial weights.")
print("STATUS: PASS")
```

```
=====
INITIAL WEIGHT CONTROL
=====
MorphBPE and BaselineBPE models have EXACTLY identical initial weights.
STATUS: PASS
```

```
# =====
# CELL 24 – SENTENCE-LEVEL DOWNSTREAM DATA VALIDATION
# =====
# Purpose:
#   Establish the exact sentence-level datasets for the controlled downstream
#   RoBERTa-Tiny experiment.
#
# Training:
#   igbo_train_corpus.txt
#
# Validation:
#   igbo_validation_corpus.txt
#
# Test:
#   igbo_test_corpus.txt
#
# IMPORTANT:
#   These files are used for downstream language-model evaluation.
#   The morphology-scoped JSON remains the source used to train MorphBPE.
# =====

from pathlib import Path
import hashlib
import re

# -----
# 1. Base directory
# -----

BASE_DIR = Path(
    "/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment"
)

DATA_DIR = BASE_DIR / "data"

print("=" * 80)
print("DOWNSTREAM DATASET DISCOVERY")
print("=" * 80)

print(f"Base directory : {BASE_DIR}")
print(f>Data directory : {DATA_DIR}")

# -----
# 2. Locate the three sentence-level files
# -----

TRAIN_FILENAME = "igbo_train_corpus.txt"
VAL_FILENAME = "igbo_validation_corpus.txt"
TEST_FILENAME = "igbo_test_corpus.txt"

def locate_file(filename):
    """
    Search Base Dir and its subdirectories for an exact filename.
    """
    matches = list(BASE_DIR.rglob(filename))

    if len(matches) == 0:
        raise FileNotFoundError(
            f"Could not find {filename} under {BASE_DIR}"
        )

    if len(matches) > 1:
        print(f"\nWARNING: Multiple copies found for {filename}:")
        for match in matches:
            print(" ", match)

        # Prefer the copy directly under BASE_DIR if present
        direct = BASE_DIR / filename
        if direct.exists():
            return direct

    return matches[0]
```

```

TRAIN_PATH = locate_file(TRAIN_FILENAME)
VAL_PATH = locate_file(VAL_FILENAME)
TEST_PATH = locate_file(TEST_FILENAME)

# -----
# 3. Report locations
# -----

print("\nLocated datasets:")
print("-" * 80)

print(f"TRAIN      : {TRAIN_PATH}")
print(f"VALIDATION  : {VAL_PATH}")
print(f"TEST       : {TEST_PATH}")

# -----
# 4. Basic file validation
# -----

for name, path in [
    ("TRAIN", TRAIN_PATH),
    ("VALIDATION", VAL_PATH),
    ("TEST", TEST_PATH),
]:
    assert path.exists(), f"{name} file does not exist: {path}"
    assert path.is_file(), f"{name} path is not a file: {path}"
    assert path.stat().st_size > 0, f"{name} file is empty: {path}"

# -----
# 5. Load sentences
# -----

def load_sentences(path):
    """
    Load one sentence per non-empty line.

    We preserve the sentence text but normalize line endings and surrounding
    whitespace.
    """
    with open(path, "r", encoding="utf-8") as f:
        sentences = [
            line.strip()
            for line in f
            if line.strip()
        ]

    return sentences

train_sentences = load_sentences(TRAIN_PATH)
validation_sentences = load_sentences(VAL_PATH)
test_sentences = load_sentences(TEST_PATH)

# -----
# 6. Dataset statistics
# -----

def sentence_statistics(sentences):
    word_counts = []
    char_counts = []

    for sentence in sentences:
        words = sentence.split()
        word_counts.append(len(words))
        char_counts.append(len(sentence))

    return {
        "sentences": len(sentences),
        "words": sum(word_counts),
        "characters": sum(char_counts),
        "avg_words": sum(word_counts) / len(word_counts),
        "avg_characters": sum(char_counts) / len(char_counts),
        "max_words": max(word_counts),
        "max_characters": max(char_counts),
    }

train_stats = sentence_statistics(train_sentences)
val_stats = sentence_statistics(validation_sentences)
test_stats = sentence_statistics(test_sentences)

```

```

print("\n" + "=" * 80)
print("SENTENCE-LEVEL DATASET STATISTICS")
print("=" * 80)

print(
    f"{'Dataset':<15}"
    f"{'Sentences':>15}"
    f"{'Words':>15}"
    f"{'Avg words':>15}"
    f"{'Max words':>15}"
)

print("-" * 80)

for name, stats in [
    ("Train", train_stats),
    ("Validation", val_stats),
    ("Test", test_stats),
]:
    print(
        f"{'name':<15}"
        f"{'stats['sentences']':>15,}"
        f"{'stats['words']':>15,}"
        f"{'stats['avg_words']':>15.2f}"
        f"{'stats['max_words']':>15,}"
    )

# -----
# 7. Check for exact sentence overlap
# -----
# No sentence should appear in more than one split.
# -----

train_set = set(train_sentences)
validation_set = set(validation_sentences)
test_set = set(test_sentences)

train_val_overlap = train_set & validation_set
train_test_overlap = train_set & test_set
val_test_overlap = validation_set & test_set

print("\n" + "=" * 80)
print("DATA SPLIT LEAKAGE CHECK")
print("=" * 80)

print(f"Train n Validation : {len(train_val_overlap):,}")
print(f"Train n Test       : {len(train_test_overlap):,}")
print(f"Validation n Test    : {len(val_test_overlap):,}")

assert len(train_val_overlap) == 0, \
    "DATA LEAKAGE: train/validation sentence overlap detected."

assert len(train_test_overlap) == 0, \
    "DATA LEAKAGE: train/test sentence overlap detected."

assert len(val_test_overlap) == 0, \
    "DATA LEAKAGE: validation/test sentence overlap detected."

# -----
# 8. Check for accidental empty/very short sentences
# -----

def short_sentence_report(sentences):
    return {
        "empty": sum(not s.strip() for s in sentences),
        "one_word": sum(len(s.split()) == 1 for s in sentences),
        "two_or_more_words": sum(len(s.split()) >= 2 for s in sentences),
    }

print("\n" + "=" * 80)
print("SENTENCE QUALITY CHECK")
print("=" * 80)

for name, sentences in [
    ("Train", train_sentences),
    ("Validation", validation_sentences),
    ("Test", test_sentences),
]:
    report = short_sentence_report(sentences)

```

```

print(f"\n{name}")
print(f"   Empty           : {report['empty']:,}")
print(f"   One word          : {report['one_word']:,}")
print(f"   Two+ words         : {report['two_or_more_words']:,}")

# -----
# 9. Display examples
# -----

print("\n" + "=" * 80)
print("SAMPLE SENTENCES")
print("=" * 80)

for name, sentences in [
    ("TRAIN", train_sentences),
    ("VALIDATION", validation_sentences),
    ("TEST", test_sentences),
]:
    print(f"\n{name}:")
    for sentence in sentences[:3]:
        print("  ", sentence)

# -----
# 10. Final status
# -----

print("\n" + "=" * 80)
print("DOWNSTREAM DATA VALIDATION")
print("=" * 80)

print("STATUS: PASS")
print()
print("Training   :", TRAIN_PATH)
print("Validation :", VAL_PATH)
print("Test       :", TEST_PATH)
print()
print("The same sentence-level datasets will be supplied to:")
print("  1. MorphBPE-8k + RoBERTa-Tiny")
print("  2. BaselineBPE-8k + RoBERTa-Tiny")
print()
print("No word-level random split will be used for downstream training.")

```



```

=====
DOWNSTREAM DATASET DISCOVERY
=====
Base directory : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment
Data directory : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/data

WARNING: Multiple copies found for igbo_train_corpus.txt:
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/igbo_train_corpus.txt
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/data/igbo_train_corpus.txt
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/Structural_MorphBPE_Experiment/data/igbo_train_corpus.txt

WARNING: Multiple copies found for igbo_validation_corpus.txt:
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/igbo_validation_corpus.txt
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/data/igbo_validation_corpus.txt
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/Structural_MorphBPE_Experiment/data/igbo_validation_corpus.txt

WARNING: Multiple copies found for igbo_test_corpus.txt:
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/igbo_test_corpus.txt
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/data/igbo_test_corpus.txt
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/Structural_MorphBPE_Experiment/data/igbo_test_corpus.txt

Located datasets:
-----
TRAIN      : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/igbo_train_corpus.txt
VALIDATION : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/igbo_validation_corpus.txt
TEST       : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/igbo_test_corpus.txt

=====
SENTENCE-LEVEL DATASET STATISTICS
=====
Dataset      Sentences      Words      Avg words      Max words
-----
Train         35,078         833,843      23.77          663
Validation    4,383          105,798      24.14          213
Test          4,388          111,897      25.50          150

=====
DATA SPLIT LEAKAGE CHECK
=====
Train n Validation : 58
Train n Test       : 60
Validation n Test   : 29

-----
AssertionError                                Traceback (most recent call last)
/tmp/ipykernel_2525/1651360852.py in <cell line: 0>()
    215
    216
--> 217 assert len(train_val_overlap) == 0, \
    218         "DATA LEAKAGE: train/validation sentence overlap detected."
    219

AssertionError: DATA LEAKAGE: train/validation sentence overlap detected.

```

Next steps: [Explain error](#)

```

# =====
# CELL 25 – INVESTIGATE SENTENCE OVERLAP
# =====
# Purpose:
#   Determine whether train/validation/test overlap represents genuine
#   duplicate sentences in the source corpus or a problematic split.
#
# IMPORTANT:
#   This cell DOES NOT modify any dataset.
# =====

from collections import Counter

# -----
# 1. Count occurrences within each split
# -----

train_counts = Counter(train_sentences)
validation_counts = Counter(validation_sentences)
test_counts = Counter(test_sentences)

# -----
# 2. Identify cross-split overlaps
# -----

train_val_overlap = set(train_counts) & set(validation_counts)
train_test_overlap = set(train_counts) & set(test_counts)
val_test_overlap = set(validation_counts) & set(test_counts)

```

```

all_three_overlap = (
    set(train_counts)
    & set(validation_counts)
    & set(test_counts)
)

# -----
# 3. Report overlap counts
# -----

print("=" * 80)
print("SENTENCE OVERLAP DIAGNOSTICS")
print("=" * 80)

print(f"Train n Validation : {len(train_val_overlap):,}")
print(f"Train n Test      : {len(train_test_overlap):,}")
print(f"Validation n Test   : {len(val_test_overlap):,}")
print(f"All three splits   : {len(all_three_overlap):,}")

# -----
# 4. Show overlapping examples
# -----

def display_overlap(name, overlap, count_a, count_b, max_examples=20):

    print("\n" + "=" * 80)
    print(name)
    print("=" * 80)

    if not overlap:
        print("No overlap found.")
        return

    print(f"Showing up to {max_examples} overlapping sentences:\n")

    for i, sentence in enumerate(
        sorted(overlap)[:max_examples],
        start=1
    ):
        print(f"[{i}] {sentence}")
        print(
            f"    Split A count: {count_a[sentence]:,} | "
            f"Split B count: {count_b[sentence]:,}"
        )

display_overlap(
    "TRAIN ↔ VALIDATION",
    train_val_overlap,
    train_counts,
    validation_counts
)

display_overlap(
    "TRAIN ↔ TEST",
    train_test_overlap,
    train_counts,
    test_counts
)

display_overlap(
    "VALIDATION ↔ TEST",
    val_test_overlap,
    validation_counts,
    test_counts
)

# -----
# 5. Determine how many sentences are actually duplicated within each split
# -----

def duplicate_statistics(counter):
    duplicated_types = sum(
        1 for count in counter.values()
        if count > 1
    )

    duplicated_instances = sum(
        count - 1
        for count in counter.values()
        if count > 1
    )

```

```

        return duplicated_types, duplicated_instances

train_dup_types, train_dup_instances = duplicate_statistics(train_counts)
val_dup_types, val_dup_instances = duplicate_statistics(validation_counts)
test_dup_types, test_dup_instances = duplicate_statistics(test_counts)

print("\n" + "=" * 80)
print("WITHIN-SPLIT DUPLICATION")
print("=" * 80)

print(
    f"Train      : {train_dup_types:,} duplicated sentence types, "
    f"{train_dup_instances:,} extra instances"
)

print(
    f"Validation : {val_dup_types:,} duplicated sentence types, "
    f"{val_dup_instances:,} extra instances"
)

print(
    f"Test       : {test_dup_types:,} duplicated sentence types, "
    f"{test_dup_instances:,} extra instances"
)

# -----
# 6. Overlap percentages
# -----

print("\n" + "=" * 80)
print("OVERLAP PERCENTAGES")
print("=" * 80)

print(
    f"Train → Validation : "
    f"{len(train_val_overlap) / len(validation_set) * 100:.3f}% "
    f"of validation types"
)

print(
    f"Train → Test      : "
    f"{len(train_test_overlap) / len(test_set) * 100:.3f}% "
    f"of test types"
)

print(
    f"Validation → Test : "
    f"{len(val_test_overlap) / len(test_set) * 100:.3f}% "
    f"of test types"
)

print("\nSTATUS: DIAGNOSTIC COMPLETE")
print("No datasets have been modified.")

```

```
[16] Mz.
      Split A count: 1 | Split B count: 2
[17] Ndịàmà Jehova na-ebipụta magazin a kemgbe afọ 1879.
      Split A count: 4 | Split B count: 5
[18] O so n' ihe e ji arụ ọrụ ikuziri ndị mmadụ Baibul n' ụwa dum.
      Split A count: 4 | Split B count: 5
[19] Ozi ọma ahụ bụ na Alaeze Chineke, nke bụ ọchịchị dị n' eluigwe, ga-eme ka ajọ omume niile kwụsị n' oge na-adighị anya, meek
      Split A count: 4 | Split B count: 5
[20] Ozi ọma dị na ya na-akasi ndị mmadụ obi.
      Split A count: 4 | Split B count: 5
```

```
=====
WITHIN-SPLIT DUPLICATION
=====
Train      : 266 duplicated sentence types, 734 extra instances
Validation : 36 duplicated sentence types, 81 extra instances
Test       : 34 duplicated sentence types, 78 extra instances
=====
```

```
OVERLAP PERCENTAGES
=====
Train → Validation : 1.348% of validation types
Train → Test       : 1.392% of test types
Validation → Test  : 0.673% of test types
=====
```

```
STATUS: DIAGNOSTIC COMPLETE
No datasets have been modified
```

```
# =====
# CELL 26 – FREEZE LEAKAGE-CONTROLLED DOWNSTREAM SPLITS
# =====

from pathlib import Path
import random
import numpy as np
import torch

SEED = 42

# Work from the sentence lists created in Cell 24.
# We preserve TRAIN and remove validation/test sentences that already occur
# in TRAIN. Then remove any remaining VALIDATION/TEST overlap from TEST.

train_final = list(train_sentences)

train_set = set(train_final)

validation_final = [
    s for s in validation_sentences
    if s not in train_set
]

validation_set = set(validation_final)

test_final = [
    s for s in test_sentences
    if s not in train_set and s not in validation_set
]

# -----
# Final leakage check
# -----

train_final_set = set(train_final)
validation_final_set = set(validation_final)
test_final_set = set(test_final)

assert not (train_final_set & validation_final_set)
assert not (train_final_set & test_final_set)
assert not (validation_final_set & test_final_set)

print("=" * 80)
print("FINAL DOWNSTREAM SPLITS")
print("=" * 80)

print(f"Original train      : {len(train_sentences):,}")
print(f"Final train         : {len(train_final):,}")
print(f"Removed from train   : {len(train_sentences) - len(train_final):,}")

print()

print(f"Original validation : {len(validation_sentences):,}")
print(f"Final validation    : {len(validation_final):,}")
print(
    f"Removed from val    : "
    f"{len(validation_sentences) - len(validation_final):,}"
)
```

```

)

print()

print(f"Original test      : {len(test_sentences):,}")
print(f"Final test         : {len(test_final):,}")
print(
    f"Removed from test   : "
    f"{len(test_sentences) - len(test_final):,}"
)

print("\nFinal overlap:")
print(
    "Train n Validation :",
    len(train_final_set & validation_final_set)
)
print(
    "Train n Test        :",
    len(train_final_set & test_final_set)
)
print(
    "Validation n Test   :",
    len(validation_final_set & test_final_set)
)

print("\nSTATUS: PASS")

```

```

=====
FINAL DOWNSTREAM SPLITS
=====
Original train      : 35,078
Final train         : 35,078
Removed from train  : 0

Original validation : 4,383
Final validation    : 4,254
Removed from val    : 129

Original test       : 4,388
Final test          : 4,256
Removed from test   : 132

Final overlap:
Train n Validation  : 0
Train n Test        : 0
Validation n Test   : 0

STATUS: PASS

```

```

# =====
# CELL 27 – BUILD CONTROLLED MLM DATASETS
# =====

from datasets import Dataset

# -----
# Convert sentence lists to Hugging Face datasets
# -----

train_dataset = Dataset.from_dict({
    "text": train_final
})

validation_dataset = Dataset.from_dict({
    "text": validation_final
})

test_dataset = Dataset.from_dict({
    "text": test_final
})

print("=" * 80)
print("DOWNSTREAM DATASETS CREATED")
print("=" * 80)

print(f"Train      : {len(train_dataset):,}")
print(f"Validation : {len(validation_dataset):,}")
print(f"Test       : {len(test_dataset):,}")

# -----
# Confirm that the raw text is identical for both experiments
# -----

assert train_dataset["text"] == Dataset.from_dict(

```

```

        {"text": train_final}
    )["text"]

    assert validation_dataset["text"] == Dataset.from_dict(
        {"text": validation_final}
    )["text"]

    assert test_dataset["text"] == Dataset.from_dict(
        {"text": test_final}
    )["text"]

    print("\nSTATUS: PASS")
    print("Both tokenizer experiments will receive identical raw sentences.")

```

```

=====
DOWNSTREAM DATASETS CREATED
=====
Train      : 35,078
Validation : 4,254
Test       : 4,256

STATUS: PASS
Both tokenizer experiments will receive identical raw sentences.

```

```

# =====
# CELL 28A – LOCATE 8K TOKENIZER ARTIFACTS
# =====

from pathlib import Path

BASE_DIR = Path(
    "/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment"
)

print("=" * 80)
print("SEARCHING FOR 8K TOKENIZER ARTIFACTS")
print("=" * 80)

# Search for relevant files
candidates = []

for pattern in [
    "*8000*",
    "*8k*",
    "*8K*",
]:
    candidates.extend(BASE_DIR.rglob(pattern))

# Remove duplicates
candidates = sorted(set(candidates))

for path in candidates:
    if path.is_file():
        print(path)

print("\n" + "=" * 80)
print("SEARCHING FOR HF TOKENIZER DIRECTORIES")
print("=" * 80)

hf_dirs = []

for path in BASE_DIR.rglob("tokenizer.json"):
    hf_dirs.append(path.parent)

if hf_dirs:
    for directory in sorted(set(hf_dirs)):
        print(directory)
else:
    print("No tokenizer.json files found.")

print("\nSTATUS: SEARCH COMPLETE")

```

```

=====
SEARCHING FOR 8K TOKENIZER ARTIFACTS
=====
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/Structural_MorphBPE_Experiment/models/b_8000_merges.txt
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/Structural_MorphBPE_Experiment/models/b_8000_vocab.json
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/Structural_MorphBPE_Experiment/models/baseline_bpe_8000.json
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/Structural_MorphBPE_Experiment/models/baseline_bpe_8000_merges.txt
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/Structural_MorphBPE_Experiment/models/baseline_bpe_8000_vocab.json
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/Structural_MorphBPE_Experiment/models/m_8000_merges.txt
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/Structural_MorphBPE_Experiment/models/m_8000_vocab.json
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/Structural_MorphBPE_Experiment/models/morph_bpe_8000.json
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/Structural_MorphBPE_Experiment/models/morph_bpe_8000_merges.txt

```

```
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/Structural_MorphBPE_Experiment/models/morph_bpe_8000_vocab.json
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/baseline_8000_merges.txt
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/baseline_8000_vocab.json
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/downstream_tokenizers/baselinebpe_8k_tokenizer.json
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/downstream_tokenizers/morphbpe_8k_tokenizer.json
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/m_8000_merges.txt
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/m_8000_vocab.json
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/morph_bpe_8000.json
```

```
=====
SEARCHING FOR HF TOKENIZER DIRECTORIES
=====
```

```
No tokenizer.json files found.
```

```
STATUS: SEARCH COMPLETE
```

```
# =====
# CELL 28 - LOAD EXISTING 8K HF TOKENIZERS
# =====

from pathlib import Path
from tokenizers import Tokenizer
from transformers import PreTrainedTokenizerFast

# -----
# 1. Exact tokenizer paths discovered in Cell 28A
# -----

BASE_DIR = Path(
    "/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment"
)

TOKENIZER_DIR = BASE_DIR / "models" / "downstream_tokenizers"

MORPH_TOKENIZER_JSON = (
    TOKENIZER_DIR / "morphbpe_8k_tokenizer.json"
)

BASELINE_TOKENIZER_JSON = (
    TOKENIZER_DIR / "baselinebpe_8k_tokenizer.json"
)

# -----
# 2. Verify files exist
# -----

print("=" * 80)
print("VERIFYING DOWNSTREAM TOKENIZER ARTIFACTS")
print("=" * 80)

assert MORPH_TOKENIZER_JSON.exists(), (
    f"Missing MorphBPE tokenizer:\n{MORPH_TOKENIZER_JSON}"
)

assert BASELINE_TOKENIZER_JSON.exists(), (
    f"Missing BaselineBPE tokenizer:\n{BASELINE_TOKENIZER_JSON}"
)

print("MorphBPE tokenizer :", MORPH_TOKENIZER_JSON)
print("Baseline tokenizer :", BASELINE_TOKENIZER_JSON)

# -----
# 3. Load raw Hugging Face Tokenizers
# -----

morph_raw_tokenizer = Tokenizer.from_file(
    str(MORPH_TOKENIZER_JSON)
)

baseline_raw_tokenizer = Tokenizer.from_file(
    str(BASELINE_TOKENIZER_JSON)
)

# -----
# 4. Wrap as Transformers fast tokenizers
# -----

morph_tokenizer = PreTrainedTokenizerFast(
    tokenizer_object=morph_raw_tokenizer,

    unk_token="<unk>",
    bos_token="<s>",
```

```

        eos_token="</s>",
        pad_token="<pad>",
        mask_token="<mask morph>",
    )

baseline_tokenizer = PreTrainedTokenizerFast(
    tokenizer_object=baseline_raw_tokenizer,

    unk_token="<unk>",
    bos_token="<s>",
    eos_token="</s>",
    pad_token="<pad>",
    mask_token="<mask morph>",
)

# -----
# 5. Vocabulary verification
# -----

print("\n" + "=" * 80)
print("VOCABULARY VERIFICATION")
print("=" * 80)

print(
    "MorphBPE vocabulary :",
    morph_tokenizer.vocab_size
)

print(
    "Baseline vocabulary :",
    baseline_tokenizer.vocab_size
)

assert morph_tokenizer.vocab_size == 8_000
assert baseline_tokenizer.vocab_size == 8_000

```

```

# -----
# 6. Special-token verification
# -----

print("\n" + "=" * 80)
print("SPECIAL TOKEN VERIFICATION")
print("=" * 80)

for name, tokenizer in [
    ("MorphBPE", morph_tokenizer),
    ("BaselineBPE", baseline_tokenizer),
]:

    print(f"\n{name}")

    print(" unk :", tokenizer.unk_token,
          "ID =", tokenizer.unk_token_id)

    print(" bos :", tokenizer.bos_token,
          "ID =", tokenizer.bos_token_id)

    print(" eos :", tokenizer.eos_token,
          "ID =", tokenizer.eos_token_id)

    print(" pad :", tokenizer.pad_token,
          "ID =", tokenizer.pad_token_id)

    print(" mask:", tokenizer.mask_token,
          "ID =", tokenizer.mask_token_id)

    assert tokenizer.unk_token_id is not None
    assert tokenizer.bos_token_id is not None
    assert tokenizer.eos_token_id is not None
    assert tokenizer.pad_token_id is not None
    assert tokenizer.mask_token_id is not None

print("\nSTATUS: PASS")
print("Both 8k tokenizer JSON files loaded successfully.")

```

```

=====
VERIFYING DOWNSTREAM TOKENIZER ARTIFACTS
=====

```

```

MorphBPE tokenizer   : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/downstream_tokenizers/morphbpe_8k_token
Baseline tokenizer   : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/downstream_tokenizers/baselinebpe_8k_to

```

```

=====
VOCABULARY VERIFICATION

```



```
=====
MorphBPE vocabulary : 8000
Baseline vocabulary : 8000
=====
```

```
SPECIAL TOKEN VERIFICATION
=====
```

```
MorphBPE
```

```
unk : <unk> ID = 0
bos : <s> ID = 1
eos : </s> ID = 2
pad : <pad> ID = 3
mask: <mask morph> ID = 4
```

```
BaselineBPE
```

```
unk : <unk> ID = 0
bos : <s> ID = 1
eos : </s> ID = 2
pad : <pad> ID = 3
mask: <mask morph> ID = 4
```

```
STATUS: PASS
```

```
Both 8k tokenizer JSON files loaded successfully.
```

```
# =====
# CELL 29 – TOKENIZER FUNCTIONALITY CHECK
# =====
```

```
print("=" * 80)
print("TOKENIZER FUNCTIONALITY CHECK")
print("=" * 80)
```

```
test_examples = [
    "Ndịàmà Jehova na-ebipụta magazin a kemgbe afọ 1879.",
    "Gịnị ka i chere?",
    "Ozi ọma dị na ya na-akasi ndị mmadụ obi.",
]
```

```
for sentence in test_examples:
```

```
    morph_encoding = morph_tokenizer(
        sentence,
        add_special_tokens=True
    )
```

```
    baseline_encoding = baseline_tokenizer(
        sentence,
        add_special_tokens=True
    )
```

```
    print("\nSentence:")
    print(sentence)
```

```
    print("\nMorphBPE:")
    print(morph_encoding.tokens())
```

```
    print("\nBaselineBPE:")
    print(baseline_encoding.tokens())
```

```
    print(
        "\nToken counts:",
        len(morph_encoding["input_ids"]),
        "vs",
        len(baseline_encoding["input_ids"])
    )
```

```
# -----
# Confirm no tokenizer produces an empty sequence
# -----
```

```
for sentence in test_examples:
```

```
    assert len(
        morph_tokenizer(sentence)["input_ids"]
    ) > 0
```

```
    assert len(
        baseline_tokenizer(sentence)["input_ids"]
    ) > 0
```

```
print("\n" + "=" * 80)
```

```
print("STATUS: PASS")
print("Both tokenizers can encode ordinary, unannotated Igbo sentences.")
```

```
=====
TOKENIZER FUNCTIONALITY CHECK
=====
```

Sentence:
Ndiàmà Jehova na-ebipụta magazin a kemgbe afọ 1879.

MorphBPE:
['N', 'dị', 'àmà', 'Jehova', 'na', '-', 'ebi', 'pụta', 'magazin', 'a', 'ke', 'm', 'g', 'be', 'afọ', '187', '9', '.']

BaselineBPE:
['N', 'dị', 'àmà', 'Je', 'hov', 'a', 'na', '-', 'ebi', 'pụta', 'magazin', 'a', 'ke', 'm', 'g', 'be', 'afọ', '187', '9', '.']

Token counts: 18 vs 20

Sentence:
Gịnị ka i chere?

MorphBPE:
['G', 'ị', 'nị', 'ka', 'i', 'c', 'here', '?']

BaselineBPE:
['G', 'ị', 'nị', 'ka', 'i', 'c', 'h', 'ere', '?']

Token counts: 8 vs 9

Sentence:
Ozi ọma dị na ya na-akasi ndị mmadụ obi.

MorphBPE:
['O', 'zi', 'ọma', 'dị', 'na', 'ya', 'na', '-', 'aka', 'si', 'ndị', 'mmadụ', 'obi', '.']

BaselineBPE:
['O', 'zi', 'ọma', 'dị', 'na', 'ya', 'na', '-', 'akasi', 'ndị', 'mmadụ', 'obi', '.']

Token counts: 14 vs 13

```
=====
STATUS: PASS
Both tokenizers can encode ordinary, unannotated Igbo sentences.
```

```
# =====
# CELL 30 – CONTROLLED ROBERTA-TINY CONFIGURATION
# =====
```

```
from transformers import RobertaConfig, RobertaForMaskedLM
```

```
# -----
# Fixed experimental constants
# -----
```

```
SEED = 42
VOCAB_SIZE = 8_000
```

```
HIDDEN_SIZE = 256
NUM_LAYERS = 4
NUM_HEADS = 4
INTERMEDIATE_SIZE = 1_024
```

```
MAX_LENGTH = 256
```

```
# -----
# Construct ONE canonical configuration
# -----
```

```
ROBERTA_CONFIG = RobertaConfig(
    vocab_size=VOCAB_SIZE,

    hidden_size=HIDDEN_SIZE,
    num_hidden_layers=NUM_LAYERS,
    num_attention_heads=NUM_HEADS,
    intermediate_size=INTERMEDIATE_SIZE,

    max_position_embeddings=MAX_LENGTH + 4,

    hidden_act="gelu",

    hidden_dropout_prob=0.1,
    attention_probs_dropout_prob=0.1,

    layer_norm_eps=1e-5,

    type_vocab_size=1,
```

```

    # Use the actual tokenizer IDs
    bos_token_id=morph_tokenizer.bos_token_id,
    eos_token_id=morph_tokenizer.eos_token_id,
    pad_token_id=morph_tokenizer.pad_token_id,

    # MLM mask
    mask_token_id=morph_tokenizer.mask_token_id,

    tie_word_embeddings=True,
)

print("=" * 80)
print("CONTROLLED ROBERTA-TINY CONFIGURATION")
print("=" * 80)

print(f"Vocabulary size      : {ROBERTA_CONFIG.vocab_size:,}")
print(f"Hidden size           : {ROBERTA_CONFIG.hidden_size}")
print(f"Transformer layers    : {ROBERTA_CONFIG.num_hidden_layers}")
print(f"Attention heads       : {ROBERTA_CONFIG.num_attention_heads}")
print(f"Intermediate size     : {ROBERTA_CONFIG.intermediate_size}")
print(f"Max positions         : {ROBERTA_CONFIG.max_position_embeddings}")

print("\nSpecial token IDs:")
print(f"  BOS  : {ROBERTA_CONFIG.bos_token_id}")
print(f"  EOS  : {ROBERTA_CONFIG.eos_token_id}")
print(f"  PAD  : {ROBERTA_CONFIG.pad_token_id}")
print(f"  MASK : {ROBERTA_CONFIG.mask_token_id}")

# -----
# Create both models from identical configuration
# -----

torch.manual_seed(SEED)

morph_model = RobertaForMaskedLM(
    ROBERTA_CONFIG
)

torch.manual_seed(SEED)

baseline_model = RobertaForMaskedLM(
    RobertaConfig.from_dict(
        ROBERTA_CONFIG.to_dict()
    )
)

# -----
# Parameter equality
# -----

def count_parameters(model):
    return sum(
        parameter.numel()
        for parameter in model.parameters()
    )

morph_parameters = count_parameters(morph_model)
baseline_parameters = count_parameters(baseline_model)

print("\n" + "=" * 80)
print("PARAMETER CONTROL")
print("=" * 80)

print(f"MorphBPE parameters : {morph_parameters:,}")
print(f"Baseline parameters : {baseline_parameters:,}")

assert morph_parameters == baseline_parameters

# -----
# Verify initial weights are identical
# -----

for name, morph_parameter in morph_model.state_dict().items():

    baseline_parameter = baseline_model.state_dict()[name]

    assert torch.equal(
        morph_parameter,
        baseline_parameter
    )

```

```

    ), f"Initial weight mismatch: {name}"

print("\nSTATUS: PASS")
print("Architecture and initial weights are identical.")

```

```

=====
CONTROLLED ROBERTA-TINY CONFIGURATION
=====
Vocabulary size      : 8,000
Hidden size         : 256
Transformer layers   : 4
Attention heads      : 4
Intermediate size    : 1024
Max positions        : 260

Special token IDs:
  BOS : 1
  EOS : 2
  PAD : 3
  MASK : 4

=====
PARAMETER CONTROL
=====
MorphBPE parameters : 5,348,672
Baseline parameters : 5,348,672

STATUS: PASS
Architecture and initial weights are identical.

```

```

import random
import numpy as np
import torch
from datasets import Dataset, DatasetDict
from transformers import DataCollatorForLanguageModeling

# -----
# Reproducibility
# -----

SEED = 42

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

# -----
# Use the leakage-controlled sentence splits
# -----

train_dataset = Dataset.from_dict({
    "text": train_final
})

validation_dataset = Dataset.from_dict({
    "text": validation_final
})

test_dataset = Dataset.from_dict({
    "text": test_final
})

raw_datasets = DatasetDict({
    "train": train_dataset,
    "validation": validation_dataset,
    "test": test_dataset
})

print("=" * 70)
print("FINAL MLM DATASET")
print("=" * 70)

for split in raw_datasets:
    print(f"{split:12s}: {len(raw_datasets[split]):,} sentences")

# -----
# Tokenization function
# -----

MAX_LENGTH = 256

def tokenize_function(examples, tokenizer):
    outputs = tokenizer(

```

```

        examples["text"],
        truncation=True,
        max_length=MAX_LENGTH,
        padding=False,
        return_attention_mask=True
    )
    # Explicitly add token_type_ids as zeros for RoBERTa-like models
    outputs["token_type_ids"] = [
        [0] * len(input_ids) for input_ids in outputs["input_ids"]
    ]
    return outputs

# -----
# Tokenize MorphBPE
# -----

morph_datasets = raw_datasets.map(
    lambda examples: tokenize_function(examples, morph_tokenizer),
    batched=True,
    remove_columns=["text"],
    desc="Tokenizing with MorphBPE"
)

# -----
# Tokenize BaselineBPE
# -----

baseline_datasets = raw_datasets.map(
    lambda examples: tokenize_function(examples, baseline_tokenizer),
    batched=True,
    remove_columns=["text"],
    desc="Tokenizing with BaselineBPE"
)

# -----
# MLM data collators
#
# Both use exactly the same masking probability.
# -----

MLM_PROBABILITY = 0.15

morph_data_collator = DataCollatorForLanguageModeling(
    tokenizer=morph_tokenizer,
    mlm=True,
    mlm_probability=MLM_PROBABILITY
)

baseline_data_collator = DataCollatorForLanguageModeling(
    tokenizer=baseline_tokenizer,
    mlm=True,
    mlm_probability=MLM_PROBABILITY
)

# -----
# Basic validation
# -----

print("\n" + "=" * 70)
print("TOKENIZED DATA")
print("=" * 70)

for name, datasets in [
    ("MorphBPE", morph_datasets),
    ("BaselineBPE", baseline_datasets)
]:
    print(f"\n{name}")

    for split in datasets:
        print(
            f" {split:12s}: "
            f"{len(datasets[split]):,} examples"
        )

print("\nMLM probability:", MLM_PROBABILITY)
print("Maximum sequence length:", MAX_LENGTH)

```

```

-----
AcceleratorError                                Traceback (most recent call last)
/tmp/ipykernel_2525/3665510561.py in <cell line: 0>()
    13 random.seed(SEED)
    14 np.random.seed(SEED)
--> 15 torch.manual_seed(SEED)
    16
    17 if torch.cuda.is_available():

```

⬆ 6 frames

```

/usr/local/lib/python3.13/dist-packages/torch/cuda/random.py in cb()
    124     for i in range(device_count()):
    125         default_generator = torch.cuda.default_generators[i]
--> 126         default_generator.manual_seed(seed)
    127
    128     _lazy_call(cb, seed_all=True)

```

AcceleratorError: CUDA error: device-side assert triggered
Search for `cudaErrorAssert` in https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART_TYPES.html for more information.
Compile with `TORCH\_USE\_CUDA\_DSA` to enable device-side assertions.

Next steps: [Explain error](#)

```

# =====
# CELL 32 - SEQUENCE LENGTH DIAGNOSTIC
# =====

def sequence_length_stats(dataset, name):
    lengths = np.array([
        len(ids) for ids in dataset["input_ids"]
    ])

    print("\n" + "=" * 70)
    print(name)
    print("=" * 70)

    print(f"Examples : {len(lengths):,}")
    print(f"Mean      : {lengths.mean():.2f}")
    print(f"Median    : {np.median(lengths):.2f}")
    print(f"P90       : {np.percentile(lengths, 90):.2f}")
    print(f"P95       : {np.percentile(lengths, 95):.2f}")
    print(f"P99       : {np.percentile(lengths, 99):.2f}")
    print(f"Maximum   : {lengths.max():,}")
    print(f">256     : {(lengths > 256).sum():,}")

    return lengths

morph_lengths = sequence_length_stats(
    morph_datasets["train"],
    "MorphBPE - Training"
)

baseline_lengths = sequence_length_stats(
    baseline_datasets["train"],
    "BaselineBPE - Training"
)

print("\n" + "=" * 70)
print("COMPARISON")
print("=" * 70)

print(
    f"MorphBPE mean length : {morph_lengths.mean():.2f}"
)

print(
    f"BaselineBPE mean length: {baseline_lengths.mean():.2f}"
)

```

```

=====
MorphBPE - Training
=====
Examples : 35,078
Mean      : 45.86
Median    : 38.00
P90       : 85.00
P95       : 109.00
P99       : 164.00
Maximum   : 256
>256     : 0

=====
BaselineBPE - Training
=====

```

```
Examples : 35,078
Mean      : 43.71
Median    : 36.00
P90       : 81.00
P95       : 104.00
P99       : 158.00
Maximum   : 256
>256      : 0
```

```
=====
COMPARISON
=====
```

```
MorphBPE mean length : 45.86
BaselineBPE mean length: 43.71
```

```
# =====
# CELL 33 – FINAL CONTROLLED MLM TRAINING CONFIGURATION
# =====

from transformers import TrainingArguments
import inspect

# -----
# Experimental controls
# -----

SEED = 42
NUM_EPOCHS = 3

TRAIN_BATCH_SIZE = 16
EVAL_BATCH_SIZE = 16

LEARNING_RATE = 5e-5
WEIGHT_DECAY = 0.01

GRADIENT_ACCUMULATION_STEPS = 1
LOGGING_STEPS = 100

# -----
# Calculate warm-up steps
# -----

NUM_TRAIN_EXAMPLES = len(morph_datasets["train"])

STEPS_PER_EPOCH = int(
    np.ceil(
        NUM_TRAIN_EXAMPLES /
        (TRAIN_BATCH_SIZE * GRADIENT_ACCUMULATION_STEPS)
    )
)

TOTAL_TRAINING_STEPS = STEPS_PER_EPOCH * NUM_EPOCHS

# Equivalent to the planned 10% warm-up
WARMUP_STEPS = int(TOTAL_TRAINING_STEPS * 0.10)

# -----
# Hardware
# -----

USE_FP16 = torch.cuda.is_available()

# -----
# Check which evaluation argument this Transformers version uses
# -----

training_args_signature = inspect.signature(
    TrainingArguments.__init__
)

if "eval_strategy" in training_args_signature.parameters:
    EVAL_ARGUMENT = "eval_strategy"
elif "evaluation_strategy" in training_args_signature.parameters:
    EVAL_ARGUMENT = "evaluation_strategy"
else:
    EVAL_ARGUMENT = None

# -----
# Common configuration
# -----

common_training_kwargs = dict(
    num_train_epochs=NUM_EPOCHS,

    per_device_train_batch_size=TRAIN_BATCH_SIZE,
    per_device_eval_batch_size=EVAL_BATCH_SIZE,
```

```

learning_rate=LEARNING_RATE,
weight_decay=WEIGHT_DECAY,

warmup_steps=WARMUP_STEPS,

gradient_accumulation_steps=GRADIENT_ACCUMULATION_STEPS,

logging_steps=LOGGING_STEPS,

save_strategy="epoch",

save_total_limit=2,

load_best_model_at_end=True,
metric_for_best_model="eval_loss",
greater_is_better=False,

fp16=USE_FP16,

seed=SEED,
data_seed=SEED,

report_to="none",

remove_unused_columns=False
)

# -----
# Add evaluation strategy using the argument supported
# by the installed Transformers version
# -----

if EVAL_ARGUMENT is not None:
    common_training_kwargs[EVAL_ARGUMENT] = "epoch"

# -----
# Output directories
# -----

OUTPUT_ROOT = (
    "/content/drive/MyDrive/Thesis/"
    "Structural_MorphBPE_Experiment/models"
)

# -----
# MorphBPE
# -----

morph_training_args = TrainingArguments(
    output_dir=(
        f"{OUTPUT_ROOT}/"
        "roberta_tiny_morphbpe_8k"
    ),
    **common_training_kwargs
)

# -----
# BaselineBPE
# -----

baseline_training_args = TrainingArguments(
    output_dir=(
        f"{OUTPUT_ROOT}/"
        "roberta_tiny_baselinebpe_8k"
    ),
    **common_training_kwargs
)

# -----
# Display configuration
# -----

print("=" * 70)
print("FINAL MLM TRAINING CONFIGURATION")
print("=" * 70)

print(f"Seed                : {SEED}")
print(f"Epochs               : {NUM_EPOCHS}")
print(f"Train examples        : {NUM_TRAIN_EXAMPLES:,}")
print(f"Steps per epoch       : {STEPS_PER_EPOCH:,}")
print(f"Total training steps  : {TOTAL_TRAINING_STEPS:,}")
print(f"Warm-up steps        : {WARMUP_STEPS:,}")
print(f"Train batch size      : {TRAIN_BATCH_SIZE}")
print(f"Evaluation batch size : {EVAL_BATCH_SIZE}")
print(f"Learning rate         : {LEARNING_RATE}")

```



```

print(f"Weight decay          : {WEIGHT_DECAY}")
print(f"Gradient accumulation : {GRADIENT_ACCUMULATION_STEPS}")
print(f"MLM probability         : {MLM_PROBABILITY}")
print(f"Maximum sequence length : {MAX_LENGTH}")
print(f"FP16                     : {USE_FP16}")
print(f"Evaluation argument     : {EVAL_ARGUMENT}")

print("\nMorphBPE output:")
print(morph_training_args.output_dir)

print("\nBaselineBPE output:")
print(baseline_training_args.output_dir)

print("\n" + "=" * 70)
print("PASS – Training configurations created.")
print("=" * 70)

```

```

=====
FINAL MLM TRAINING CONFIGURATION
=====
Seed                : 42
Epochs             : 3
Train examples      : 35,078
Steps per epoch     : 2,193
Total training steps : 6,579
Warm-up steps       : 657
Train batch size    : 16
Evaluation batch size : 16
Learning rate       : 5e-05
Weight decay        : 0.01
Gradient accumulation : 1
MLM probability     : 0.15
Maximum sequence length : 256
FP16                : True
Evaluation argument  : eval_strategy

MorphBPE output:
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/roberta_tiny_morphbpe_8k

BaselineBPE output:
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/roberta_tiny_baselinebpe_8k

=====
PASS – Training configurations created.
=====

```

```

# =====
# CELL 34 – EXPERIMENTAL CONTROL CHECK
# =====

print("=" * 70)
print("EXPERIMENTAL CONTROL CHECK")
print("=" * 70)

checks = {
    "Vocabulary size": (
        morph_tokenizer.vocab_size,
        baseline_tokenizer.vocab_size
    ),

    "Hidden size": (
        morph_model.config.hidden_size,
        baseline_model.config.hidden_size
    ),

    "Layers": (
        morph_model.config.num_hidden_layers,
        baseline_model.config.num_hidden_layers
    ),

    "Attention heads": (
        morph_model.config.num_attention_heads,
        baseline_model.config.num_attention_heads
    ),

    "Intermediate size": (
        morph_model.config.intermediate_size,
        baseline_model.config.intermediate_size
    ),

    "Train batch size": (
        morph_training_args.per_device_train_batch_size,
        baseline_training_args.per_device_train_batch_size
    ),

    "Eval batch size": (
        morph_training_args.per_device_eval_batch_size,

```

```

        baseline_training_args.per_device_eval_batch_size
    ),

    "Learning rate": (
        morph_training_args.learning_rate,
        baseline_training_args.learning_rate
    ),

    "Epochs": (
        morph_training_args.num_train_epochs,
        baseline_training_args.num_train_epochs
    ),

    "Weight decay": (
        morph_training_args.weight_decay,
        baseline_training_args.weight_decay
    ),

    "Seed": (
        morph_training_args.seed,
        baseline_training_args.seed
    ),

    "Data seed": (
        morph_training_args.data_seed,
        baseline_training_args.data_seed
    ),
}

all_equal = True

for name, (morph_value, baseline_value) in checks.items():
    same = morph_value == baseline_value

    print(
        f"{name:25s}: "
        f"MorphBPE={morph_value} | "
        f"BaselineBPE={baseline_value} | "
        f"{'PASS' if same else 'FAIL'}"
    )

    if not same:
        all_equal = False

# Parameter counts
morph_params = sum(
    p.numel() for p in morph_model.parameters()
)

baseline_params = sum(
    p.numel() for p in baseline_model.parameters()
)

print(
    f"\nParameter count: "
    f"MorphBPE={morph_params:,} | "
    f"BaselineBPE={baseline_params:,}"
)

if morph_params != baseline_params:
    all_equal = False

print("\n" + "=" * 70)

if all_equal:
    print("PASS – Controlled experimental configuration is identical.")
else:
    print("FAIL – Experimental controls are not identical.")

```

```

=====
EXPERIMENTAL CONTROL CHECK
=====
Vocabulary size      : MorphBPE=8000 | BaselineBPE=8000 | PASS
Hidden size         : MorphBPE=256 | BaselineBPE=256 | PASS
Layers              : MorphBPE=4 | BaselineBPE=4 | PASS
Attention heads     : MorphBPE=4 | BaselineBPE=4 | PASS
Intermediate size   : MorphBPE=1024 | BaselineBPE=1024 | PASS
Train batch size    : MorphBPE=16 | BaselineBPE=16 | PASS
Eval batch size     : MorphBPE=16 | BaselineBPE=16 | PASS
Learning rate       : MorphBPE=5e-05 | BaselineBPE=5e-05 | PASS
Epochs             : MorphBPE=3 | BaselineBPE=3 | PASS
Weight decay        : MorphBPE=0.01 | BaselineBPE=0.01 | PASS
Seed                : MorphBPE=42 | BaselineBPE=42 | PASS
Data seed           : MorphBPE=42 | BaselineBPE=42 | PASS

Parameter count: MorphBPE=5,348,672 | BaselineBPE=5,348,672

```

```
=====
PASS – Controlled experimental configuration is identical.
```

```
# =====
# CELL 35 – MORPHBPE MLM TRAINER
# =====

from transformers import Trainer

# -----
# Reconfirm the MorphBPE model exists
# -----

assert morph_model is not None, "MorphBPE model is not defined."
assert morph_tokenizer is not None, "MorphBPE tokenizer is not defined."

# -----
# Create Trainer
# -----

morph_trainer = Trainer(
    model=morph_model,

    args=morph_training_args,

    train_dataset=morph_datasets["train"],
    eval_dataset=morph_datasets["validation"],

    data_collator=morph_data_collator,

    processing_class=morph_tokenizer
)

# -----
# Configuration summary
# -----

print("=" * 70)
print("MORPHBPE MLM TRAINER")
print("=" * 70)

print("Model           : RoBERTa-Tiny")
print("Tokenizer          : MorphBPE")
print("Vocabulary          : 8,000")
print("Training examples   : ", f"{len(morph_datasets['train']):,}")
print("Validation examples : ", f"{len(morph_datasets['validation']):,}")
print("Epochs             : ", NUM_EPOCHS)
print("Batch size          : ", TRAIN_BATCH_SIZE)
print("Learning rate        : ", LEARNING_RATE)
print("MLM probability      : ", MLM_PROBABILITY)
print("Seed                : ", SEED)

print("\nPASS – MorphBPE Trainer created successfully.")
```

```
=====
MORPHBPE MLM TRAINER
=====
Model           : RoBERTa-Tiny
Tokenizer        : MorphBPE
Vocabulary       : 8,000
Training examples : 35,078
Validation examples : 4,254
Epochs          : 3
Batch size       : 16
Learning rate    : 5e-05
MLM probability  : 0.15
Seed            : 42

PASS – MorphBPE Trainer created successfully.
```

```
# =====
# CELL 36 – BASELINEBPE MLM TRAINER
# =====

from transformers import Trainer

# -----
# Reconfirm the BaselineBPE model and tokenizer
# -----

assert baseline_model is not None, "BaselineBPE model is not defined."
assert baseline_tokenizer is not None, "BaselineBPE tokenizer is not defined."

# -----
# Create Trainer
# -----
```

```

baseline_trainer = Trainer(
    model=baseline_model,

    args=baseline_training_args,

    train_dataset=baseline_datasets["train"],
    eval_dataset=baseline_datasets["validation"],

    data_collator=baseline_data_collator,

    processing_class=baseline_tokenizer
)

# -----
# Configuration summary
# -----

print("=" * 70)
print("BASELINEBPE MLM TRAINER")
print("=" * 70)

print("Model           : RoBERTa-Tiny")
print("Tokenizer         : BaselineBPE")
print("Vocabulary         : 8,000")
print("Training examples  :", f"{len(baseline_datasets['train']):,}")
print("Validation examples:", f"{len(baseline_datasets['validation']):,}")
print("Epochs            :", NUM_EPOCHS)
print("Batch size         :", TRAIN_BATCH_SIZE)
print("Learning rate       :", LEARNING_RATE)
print("MLM probability     :", MLM_PROBABILITY)
print("Seed               :", SEED)

print("\nPASS - BaselineBPE Trainer created successfully.")

```

```

=====
BASELINEBPE MLM TRAINER
=====
Model           : RoBERTa-Tiny
Tokenizer         : BaselineBPE
Vocabulary         : 8,000
Training examples : 35,078
Validation examples: 4,254
Epochs            : 3
Batch size         : 16
Learning rate       : 5e-05
MLM probability     : 0.15
Seed               : 42

PASS - BaselineBPE Trainer created successfully.

```

```

# =====
# CELL 37 - FINAL PRE-TRAINING SANITY CHECK
# =====

import os
import torch

print("=" * 70)
print("FINAL PRE-TRAINING SANITY CHECK")
print("=" * 70)

# -----
# 1. GPU CHECK
# -----

print("\n[1] HARDWARE")

if torch.cuda.is_available():
    print("CUDA available : YES")
    print("GPU           :", torch.cuda.get_device_name(0))
else:
    print("CUDA available : NO")
    print("WARNING: Training will run on CPU.")

# -----
# 2. TOKENIZER CHECK
# -----

print("\n[2] TOKENIZERS")

print(
    f"MorphBPE vocab size : "
    f"{morph_tokenizer.vocab_size:,}"
)

```

```

print(
    f"BaselineBPE vocab size: "
    f"{baseline_tokenizer.vocab_size:,}"
)

assert morph_tokenizer.vocab_size == 8000
assert baseline_tokenizer.vocab_size == 8000

print("PASS – Both tokenizers have exactly 8,000 tokens.")

# -----
# 3. MODEL CHECK
# -----

print("\n[3] MODELS")

morph_params = sum(
    p.numel() for p in morph_model.parameters()
)

baseline_params = sum(
    p.numel() for p in baseline_model.parameters()
)

print(f"MorphBPE parameters : {morph_params:,}")
print(f"BaselineBPE parameters: {baseline_params:,}")

assert morph_params == baseline_params

print("PASS – Parameter counts are identical.")

# -----
# 4. DATASET CHECK
# -----

print("\n[4] DATASETS")

print(
    f"MorphBPE train      : "
    f"{len(morph_datasets['train']):,}"
)

print(
    f"BaselineBPE train    : "
    f"{len(baseline_datasets['train']):,}"
)

print(
    f"MorphBPE validation  : "
    f"{len(morph_datasets['validation']):,}"
)

print(
    f"BaselineBPE validation: "
    f"{len(baseline_datasets['validation']):,}"
)

print(
    f"MorphBPE test        : "
    f"{len(morph_datasets['test']):,}"
)

print(
    f"BaselineBPE test      : "
    f"{len(baseline_datasets['test']):,}"
)

assert (
    len(morph_datasets["train"])
    == len(baseline_datasets["train"])
)

assert (
    len(morph_datasets["validation"])
    == len(baseline_datasets["validation"])
)

assert (
    len(morph_datasets["test"])
    == len(baseline_datasets["test"])
)

print("PASS – Both conditions use identical sentence splits.")

# -----
# 5. TRAINING CONFIGURATION CHECK

```

```

# -----

print("\n[5] TRAINING CONTROLS")

configuration_checks = {
    "Epochs":
        morph_training_args.num_train_epochs
        == baseline_training_args.num_train_epochs,

    "Train batch size":
        morph_training_args.per_device_train_batch_size
        == baseline_training_args.per_device_train_batch_size,

    "Eval batch size":
        morph_training_args.per_device_eval_batch_size
        == baseline_training_args.per_device_eval_batch_size,

    "Learning rate":
        morph_training_args.learning_rate
        == baseline_training_args.learning_rate,

    "Weight decay":
        morph_training_args.weight_decay
        == baseline_training_args.weight_decay,

    "Warmup steps":
        morph_training_args.warmup_steps
        == baseline_training_args.warmup_steps,

    "Gradient accumulation":
        morph_training_args.gradient_accumulation_steps
        == baseline_training_args.gradient_accumulation_steps,

    "Seed":
        morph_training_args.seed
        == baseline_training_args.seed,

    "Data seed":
        morph_training_args.data_seed
        == baseline_training_args.data_seed,
}

all_controls_pass = True

for name, passed in configuration_checks.items():
    print(
        f"{name:25s}: "
        f"'PASS' if passed else 'FAIL'"
    )

    if not passed:
        all_controls_pass = False

# -----
# 6. OUTPUT DIRECTORIES
# -----

print("\n[6] OUTPUT DIRECTORIES")

print(
    "MorphBPE  :",
    morph_training_args.output_dir
)

print(
    "BaselineBPE:",
    baseline_training_args.output_dir
)

# -----
# 7. TEST SET IS NOT PASSED TO TRAINER
# -----

print("\n[7] TEST-SET ISOLATION")

print(
    "MorphBPE Trainer training dataset :",
    len(morph_trainer.train_dataset)
)

print(
    "MorphBPE Trainer eval dataset      :",
    len(morph_trainer.eval_dataset)
)

print(

```

```

    "BaselineBPE Trainer training dataset:",
    len(baseline_trainer.train_dataset)
)

print(
    "BaselineBPE Trainer eval dataset    :",
    len(baseline_trainer.eval_dataset)
)

assert "test" not in morph_trainer.train_dataset.column_names
assert "test" not in baseline_trainer.train_dataset.column_names

print(
    "PASS – Test data is not supplied to either Trainer."
)

# -----
# FINAL RESULT
# -----

print("\n" + "=" * 70)

if all_controls_pass:
    print("PASS – EXPERIMENT READY FOR TRAINING")
else:
    print("FAIL – DO NOT TRAIN")
    print("Resolve the failed control(s) first.")

print("=" * 70)

```

=====

FINAL PRE-TRAINING SANITY CHECK

=====

[1] HARDWARE

```

CUDA available : YES
GPU            : Tesla T4

```

[2] TOKENIZERS

```

MorphBPE vocab size   : 8,000
BaselineBPE vocab size: 8,000
PASS – Both tokenizers have exactly 8,000 tokens.

```

[3] MODELS

```

MorphBPE parameters   : 5,348,672
BaselineBPE parameters: 5,348,672
PASS – Parameter counts are identical.

```

[4] DATASETS

```

MorphBPE train        : 35,078
BaselineBPE train     : 35,078
MorphBPE validation   : 4,254
BaselineBPE validation: 4,254
MorphBPE test         : 4,256
BaselineBPE test      : 4,256
PASS – Both conditions use identical sentence splits.

```

[5] TRAINING CONTROLS

```

Epochs              : PASS
Train batch size     : PASS
Eval batch size      : PASS
Learning rate        : PASS
Weight decay         : PASS
Warmup steps         : PASS
Gradient accumulation: PASS
Seed                 : PASS
Data seed            : PASS

```

[6] OUTPUT DIRECTORIES

```

MorphBPE   : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/roberta_tiny_morphbpe_8k
BaselineBPE: /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/roberta_tiny_baselinebpe_8k

```

[7] TEST-SET ISOLATION

```

MorphBPE Trainer training dataset : 35078
MorphBPE Trainer eval dataset     : 4254
BaselineBPE Trainer training dataset: 35078
BaselineBPE Trainer eval dataset   : 4254
PASS – Test data is not supplied to either Trainer.

```

```

=====
PASS – EXPERIMENT READY FOR TRAINING
=====

```

```

# =====
# CELL 38C – FRESH CUDA SANITY TEST
# =====

```

```
import torch
```

```
print("=" * 70)
```

```

print("FRESH CUDA SANITY TEST")
print("=" * 70)

print("PyTorch version :", torch.__version__)
print("CUDA available :", torch.cuda.is_available())

if not torch.cuda.is_available():
    raise RuntimeError("CUDA is not available.")

print("GPU          :", torch.cuda.get_device_name(0))
print("CUDA version  :", torch.version.cuda)

# -----
# Simple CUDA tensor
# -----

x = torch.tensor(
    [0, 1, 2, 3],
    dtype=torch.long,
    device="cuda"
)

print("\nCUDA tensor:", x)

# -----
# Simple embedding
# -----

embedding = torch.nn.Embedding(
    num_embeddings=8000,
    embedding_dim=256
).cuda()

y = embedding(x)

print("Embedding shape:", tuple(y.shape))

print("\n" + "=" * 70)
print("PASS – CUDA RUNTIME IS HEALTHY")
print("=" * 70)

```

```

=====
FRESH CUDA SANITY TEST
=====
PyTorch version : 2.11.0+cu128
CUDA available  : True
GPU             : Tesla T4
CUDA version    : 12.8

CUDA tensor: tensor([0, 1, 2, 3], device='cuda:0')
Embedding shape: (4, 256)

=====
PASS – CUDA RUNTIME IS HEALTHY
=====

```

```

# =====
# CELL 38F – MORPHBPE MODEL CUDA TRANSFER TEST
# =====

import torch

print("=" * 70)
print("MORPHBPE MODEL CUDA TRANSFER TEST")
print("=" * 70)

print("Model class:", type(morph_model).__name__)
print("Model vocab size:", morph_model.config.vocab_size)
print("Model parameters:", sum(p.numel() for p in morph_model.parameters()))

# Check model is currently on CPU
model_device_before = next(morph_model.parameters()).device
print("Device before transfer:", model_device_before)

# Clear CUDA cache
torch.cuda.empty_cache()

# Move ONLY the model to GPU
print("\nMoving MorphBPE model to CUDA...")

morph_model = morph_model.to("cuda")

model_device_after = next(morph_model.parameters()).device

print("Device after transfer:", model_device_after)

```



```
# Simple tensor through the model's embedding layer
test_ids = torch.tensor(
    [[1, 16, 7428, 2]],
    dtype=torch.long,
    device="cuda"
)

print("\nTest input IDs:", test_ids)
print("Test input range:",
      test_ids.min().item(),
      "to",
      test_ids.max().item())

with torch.no_grad():
    embedding_output = morph_model.roberta.embeddings.word_embeddings(test_ids)

print("Embedding output shape:", tuple(embedding_output.shape))

print("\n" + "=" * 70)
print("PASS – MORPHBPE MODEL TRANSFERS TO CUDA")
print("=" * 70)
```

```
=====
MORPHBPE MODEL CUDA TRANSFER TEST
=====
Model class: RobertaForMaskedLM
Model vocab size: 8000
Model parameters: 5348672
Device before transfer: cuda:0

Moving MorphBPE model to CUDA...
Device after transfer: cuda:0

Test input IDs: tensor([[ 1, 16, 7428, 2]], device='cuda:0')
Test input range: 1 to 7428
Embedding output shape: (1, 4, 256)

=====
PASS – MORPHBPE MODEL TRANSFERS TO CUDA
=====
```

```
# =====
# CELL 38G – ROBERTA FULL FORWARD-PASS DIAGNOSTIC
# =====

import torch

print("=" * 70)
print("ROBERTA FULL FORWARD-PASS DIAGNOSTIC")
print("=" * 70)

# -----
# 1. Create a very small controlled input
# -----

test_input_ids = torch.tensor(
    [
        [1, 16, 7428, 2970, 2],
        [1, 25, 183, 901, 2],
    ],
    dtype=torch.long,
    device="cuda"
)

test_attention_mask = torch.ones_like(test_input_ids)

print("Input shape :", tuple(test_input_ids.shape))
print("Mask shape :", tuple(test_attention_mask.shape))
print("Input range :", test_input_ids.min().item(), "to", test_input_ids.max().item())

# -----
# 2. Confirm model is on CUDA
# -----

print("Model device:", next(morph_model.parameters()).device)

# -----
# 3. Run the complete RoBERTa forward pass
# -----

morph_model.eval()

print("\nRunning full RoBERTa forward pass...")

with torch.no_grad():
    outputs = morph_model(
        input_ids=test_input_ids,
```

```

        attention_mask=test_attention_mask
    )

print("\nForward pass completed successfully.")

print("Logits shape:", tuple(outputs.logits.shape))
print("Expected shape:",
      (2, 5, morph_model.config.vocab_size))

print("Logits finite:",
      torch.isfinite(outputs.logits).all().item())

print("\n" + "=" * 70)
print("PASS – FULL ROBERTA FORWARD PASS WORKS")
print("=" * 70)

```

```

=====
ROBERTA FULL FORWARD-PASS DIAGNOSTIC
=====
Input shape : (2, 5)
Mask shape  : (2, 5)
Input range : 1 to 7428
Model device: cuda:0

Running full RoBERTa forward pass...

Forward pass completed successfully.
Logits shape: (2, 5, 8000)
Expected shape: (2, 5, 8000)
Logits finite: True

=====
PASS – FULL ROBERTA FORWARD PASS WORKS
=====

```

```

# =====
# CELL 38H – ACTUAL MLM BATCH FORWARD-PASS TEST
# =====

import torch

print("=" * 70)
print("ACTUAL MLM BATCH FORWARD-PASS TEST")
print("=" * 70)

# -----
# 1. Get two real training examples
# -----

samples = [
    morph_datasets["train"][0],
    morph_datasets["train"][1],
]

# -----
# 2. Apply the actual MorphBPE MLM collator
# -----

batch = morph_data_collator(samples)

print("CPU batch:")
for key, value in batch.items():
    print(
        f"    {key:15s}"
        f" shape={tuple(value.shape)}"
        f" dtype={value.dtype}"
    )

# -----
# 3. Validate values before CUDA transfer
# -----

input_ids = batch["input_ids"]
attention_mask = batch["attention_mask"]
labels = batch["labels"]

print("\nInput IDs:")
print("  min:", input_ids.min().item())
print("  max:", input_ids.max().item())

assert input_ids.min().item() >= 0
assert input_ids.max().item() < VOCAB_SIZE

print("  PASS – input IDs valid")

print("\nLabels:")

```

```

valid_labels = labels[labels != -100]

if valid_labels.numel() > 0:
    print(" min:", valid_labels.min().item())
    print(" max:", valid_labels.max().item())

    assert valid_labels.min().item() >= 0
    assert valid_labels.max().item() < VOCAB_SIZE

print(" PASS – MLM labels valid")

print("\nAttention mask:")
print(" unique:", torch.unique(attention_mask).tolist())

assert torch.all(
    (attention_mask == 0) | (attention_mask == 1)
)

print(" PASS – attention mask valid")

# -----
# 4. Transfer each tensor individually
# -----

print("\nTesting individual CUDA transfers...")

input_ids_gpu = input_ids.to("cuda")
print(" input_ids      → PASS")

attention_mask_gpu = attention_mask.to("cuda")
print(" attention_mask → PASS")

labels_gpu = labels.to("cuda")
print(" labels          → PASS")

# -----
# 5. Run actual MLM forward pass
# -----

print("\nRunning actual MLM forward pass...")

morph_model.eval()

with torch.no_grad():
    outputs = morph_model(
        input_ids=input_ids_gpu,
        attention_mask=attention_mask_gpu,
        labels=labels_gpu
    )

print("\nMLM forward pass completed.")

print("Loss:", outputs.loss.item())
print("Logits shape:", tuple(outputs.logits.shape))
print("Expected:",
      (input_ids.shape[0],
       input_ids.shape[1],
       VOCAB_SIZE))

assert torch.isfinite(outputs.loss)
assert torch.isfinite(outputs.logits).all()

print("\n" + "=" * 70)
print("PASS – ACTUAL MLM BATCH FORWARD PASS WORKS")
print("=" * 70)

```

```

=====
ACTUAL MLM BATCH FORWARD-PASS TEST
=====
CPU batch:
input_ids      shape=(2, 68) dtype=torch.int64
attention_mask shape=(2, 68) dtype=torch.int64
token_type_ids shape=(2, 68) dtype=torch.int64
labels         shape=(2, 68) dtype=torch.int64

Input IDs:
min: 3
max: 7429
PASS – input IDs valid

Labels:
min: 89
max: 6236
PASS – MLM labels valid

Attention mask:
unique: [0, 1]

```

```

PASS - attention mask valid

Testing individual CUDA transfers...
input_ids      → PASS
attention_mask → PASS
labels         → PASS

Running actual MLM forward pass...

MLM forward pass completed.
Loss: 8.977537155151367
Logits shape: (2, 68, 8000)
Expected: (2, 68, 8000)

=====
PASS - ACTUAL MLM BATCH FORWARD PASS WORKS
=====

```

```

# =====
# CELL 38I - ONE-STEP TRAINER DIAGNOSTIC
# =====

import torch

print("=" * 70)
print("ONE-STEP TRAINER DIAGNOSTIC")
print("=" * 70)

# -----
# Create a fresh model with the same experimental configuration
# -----

torch.manual_seed(SEED)

test_model = RobertaForMaskedLM(ROBERTA_CONFIG)

print("Model created.")
print("Parameters:",
      sum(p.numel() for p in test_model.parameters()))

# -----
# Create minimal training arguments
# -----

test_output_dir = str(BASE_DIR / "models" / "debug_trainer_one_step")

test_training_args = TrainingArguments(
    output_dir=test_output_dir,

    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,

    max_steps=1,

    learning_rate=5e-5,
    weight_decay=0.01,

    logging_steps=1,

    save_strategy="no",
    eval_strategy="no",

    seed=SEED,
    data_seed=SEED,

    fp16=torch.cuda.is_available(),

    report_to="none",

    remove_unused_columns=False,
)

# -----
# Create a fresh Trainer
# -----

test_trainer = Trainer(
    model=test_model,
    args=test_training_args,
    train_dataset=morph_datasets["train"],
    data_collator=morph_data_collator,
    processing_class=morph_tokenizer,
)

print("Trainer created.")
print("Training examples:", len(morph_datasets["train"]))

```

```
# -----
# Run exactly ONE optimizer step
# -----

print("\nRunning exactly ONE Trainer training step...")

train_result = test_trainer.train()

print("\nTrainer completed successfully.")

print("Training loss:",
      train_result.training_loss)

print("\n" + "=" * 70)
print("PASS – TRAINER ONE-STEP TEST WORKS")
print("=" * 70)
```

```
=====
ONE-STEP TRAINER DIAGNOSTIC
=====
Model created.
Parameters: 5348672
Trainer created.
Training examples: 35078

Running exactly ONE Trainer training step...
[1/1 00:00, Epoch 0/1]

Step Training Loss
-----
1          9.087891

Trainer completed successfully.
Training loss: 9.087890625

=====
PASS – TRAINER ONE-STEP TEST WORKS
=====
```

```
# =====
# CELL 39 – TRAIN MORPHBPE ROBERTA-TINY
# =====

print("=" * 70)
print("TRAINING MORPHBPE ROBERTA-TINY")
print("=" * 70)

print("Training examples :", len(morph_datasets["train"]))
print("Validation examples:", len(morph_datasets["validation"]))
print("Epochs           :", morph_training_args.num_train_epochs)
print("Batch size         :", morph_training_args.per_device_train_batch_size)
print("Learning rate       :", morph_training_args.learning_rate)
print("Output directory    :", morph_training_args.output_dir)

print("\nStarting MorphBPE training...\n")

morph_train_result = morph_trainer.train()

print("\n" + "=" * 70)
print("MORPHBPE TRAINING COMPLETED")
print("=" * 70)

print("\nTraining loss:",
      morph_train_result.training_loss)

print("\nTraining metrics:")
for key, value in morph_train_result.metrics.items():
    print(f" {key}: {value}")
```

```
=====
TRAINING MORPHBPE ROBERTA-TINY
=====
Training examples : 35078
Validation examples: 4254
Epochs           : 3
Batch size        : 16
Learning rate      : 5e-05
Output directory   : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/roberta_tiny_morphbpe_8k
```

Starting MorphBPE training...

 [6579/6579 05:27, Epoch 3/3]

Epoch	Training Loss	Validation Loss
1	5.358868	5.322700
2	5.196640	5.143006
3	5.123850	5.068519

Writing model shards: 100% 1/1 [00:00<00:00, 9.73it/s]

Writing model shards: 100% 1/1 [00:00<00:00, 11.24it/s]

Writing model shards: 100% 1/1 [00:00<00:00, 11.37it/s]

[transformers] There were missing keys in the checkpoint model loaded: ['lm_head.decoder.weight', 'lm_head.decoder.bias'].

=====

MORPHBPE TRAINING COMPLETED

Training loss: 5.485094994972462

Training metrics:

```

train_runtime: 328.029
train_samples_per_second: 320.807
train_steps_per_second: 20.056
total_flos: 251301306961152.0
train_loss: 5.485094994972462
epoch: 3.0
```

```
# =====
# CELL 40B - FINAL MORPHBPE CHECKPOINT VERIFICATION
# =====

import torch
from pathlib import Path
from transformers import RobertaForMaskedLM

print("=" * 70)
print("FINAL MORPHBPE CHECKPOINT VERIFICATION")
print("=" * 70)

MORPH_OUTPUT_DIR = Path(
    "/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/"
    "models/roberta_tiny_morphbpe_8k"
)

FINAL_CHECKPOINT = MORPH_OUTPUT_DIR / "checkpoint-6579"

print("Checkpoint:", FINAL_CHECKPOINT)

assert FINAL_CHECKPOINT.exists()
assert (FINAL_CHECKPOINT / "model.safetensors").exists()

print(" PASS - checkpoint and model weights exist")

# -----
# 1. Reload model
# -----

reloaded_morph_model = RobertaForMaskedLM.from_pretrained(
    str(FINAL_CHECKPOINT)
)

print("\nModel reloaded successfully.")

# -----
# 2. Configuration
# -----

config = reloaded_morph_model.config

print("\nConfiguration:")
print(" Vocabulary size :", config.vocab_size)
print(" Hidden size      :", config.hidden_size)
print(" Layers           :", config.num_hidden_layers)
```

```

print(" Attention heads :", config.num_attention_heads)
print(" Intermediate      :", config.intermediate_size)
print(" Max positions      :", config.max_position_embeddings)

assert config.vocab_size == 8000
assert config.hidden_size == 256
assert config.num_hidden_layers == 4
assert config.num_attention_heads == 4
assert config.intermediate_size == 1024
assert config.max_position_embeddings == 260

print(" PASS – configuration is correct")

# -----
# 3. Parameter count
# -----

parameter_count = sum(
    p.numel() for p in reloaded_morph_model.parameters()
)

expected_parameter_count = 5_348_672

print("\nParameter count:", parameter_count)
print("Expected count :", expected_parameter_count)

assert parameter_count == expected_parameter_count

print(" PASS – parameter count is correct")

# -----
# 4. Tokenizer
# -----

print("\nTokenizer vocabulary size:", morph_tokenizer.vocab_size)

assert morph_tokenizer.vocab_size == 8000

print(" PASS – tokenizer vocabulary is 8,000")

# -----
# 5. GPU inference test
# -----

reloaded_morph_model = reloaded_morph_model.to("cuda")
reloaded_morph_model.eval()

validation_sample = morph_datasets["validation"][0]

validation_batch = morph_data_collator(
    [validation_sample]
)

validation_batch = {
    key: value.to("cuda")
    for key, value in validation_batch.items()
}

print("\nRunning validation MLM forward pass...")

with torch.no_grad():
    validation_output = reloaded_morph_model(
        input_ids=validation_batch["input_ids"],
        attention_mask=validation_batch["attention_mask"],
        labels=validation_batch["labels"]
    )

validation_loss = validation_output.loss.item()

print("Validation batch loss:", validation_loss)
print("Logits shape:", tuple(validation_output.logits.shape))

assert torch.isfinite(validation_output.loss)
assert torch.isfinite(validation_output.logits).all()
assert validation_output.logits.shape[-1] == 8000

print(" PASS – checkpoint produces valid MLM output")

# -----
# 6. Final status
# -----

print("\n" + "=" * 70)
print("PASS – MORPHBPE CHECKPOINT FULLY VERIFIED")
print("=" * 70)

```

```
print("\nFinal training validation loss : 5.068519")
print("Checkpoint parameter count      :", parameter_count)
print("Checkpoint max positions         :", config.max_position_embeddings)
print("Checkpoint vocabulary             :", config.vocab_size)
print("Checkpoint path                   :", FINAL_CHECKPOINT)
```

```
=====
L MORPHBPE CHECKPOINT VERIFICATION
=====
```

```
kpoint: /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/roberta_tiny_morphbpe_8k/checkpoint-6579
SS - checkpoint and model weights exist
```

```
ng weights: 100%                                74/74 [00:00<00:00, 1288.87it/s]
```

```
l reloaded successfully.
```

```
figuration:
vocabulary size : 8000
hidden size     : 256
layers          : 4
attention heads : 4
intermediate    : 1024
max positions   : 260
SS - configuration is correct
```

```
parameter count: 5348672
checked count : 5348672
SS - parameter count is correct
```

```
tokenizer vocabulary size: 8000
SS - tokenizer vocabulary is 8,000
```

```
performing validation MLM forward pass...
validation batch loss: 4.7427592277526855
tokens shape: (1, 37, 8000)
SS - checkpoint produces valid MLM output
```

```
=====
- MORPHBPE CHECKPOINT FULLY VERIFIED
=====
```

```
l training validation loss : 5.068519
kpoint parameter count      : 5348672
kpoint max positions        : 260
kpoint vocabulary          : 8000
kpoint path                 : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/roberta_tiny_morphbpe_8k/checkpoint-6579
```

```
# =====
# CELL 41 - INITIALIZE BASELINEBPE ROBERTA-TINY
# =====
```

```
import torch
```

```
print("=" * 70)
print("INITIALIZING BASELINEBPE ROBERTA-TINY")
print("=" * 70)
```

```
torch.manual_seed(SEED)
```

```
baseline_model = RobertaForMaskedLM(ROBERTA_CONFIG)
```

```
baseline_parameter_count = sum(
    p.numel() for p in baseline_model.parameters()
)
```

```
print("Vocabulary size :", baseline_model.config.vocab_size)
print("Hidden size      :", baseline_model.config.hidden_size)
print("Layers           :", baseline_model.config.num_hidden_layers)
print("Attention heads   :", baseline_model.config.num_attention_heads)
print("Intermediate      :", baseline_model.config.intermediate_size)
print("Max positions     :", baseline_model.config.max_position_embeddings)
print("Parameters        :", baseline_parameter_count)
```

```
assert baseline_model.config.vocab_size == 8000
assert baseline_model.config.max_position_embeddings == 260
assert baseline_parameter_count == 5_348_672
```

```
print("\nPASS - BaselineBPE model initialized correctly")
```

```
=====
INITIALIZING BASELINEBPE ROBERTA-TINY
=====
```

```
Vocabulary size : 8000
Hidden size     : 256
Layers          : 4
Attention heads : 4
Intermediate    : 1024
Max positions   : 260
```


Parameters : 5348672

PASS – BaselineBPE model initialized correctly

```
# =====
# CELL 42 – BASELINEBPE ONE-STEP TRAINER TEST
# =====

import torch
from transformers import Trainer

print("=" * 70)
print("BASELINEBPE ONE-STEP TRAINER TEST")
print("=" * 70)

# -----
# 1. Verify BaselineBPE configuration
# -----

print("Vocabulary size :", baseline_model.config.vocab_size)
print("Max positions : ", baseline_model.config.max_position_embeddings)

baseline_parameter_count = sum(
    p.numel() for p in baseline_model.parameters()
)

print("Parameters : ", baseline_parameter_count)

assert baseline_model.config.vocab_size == 8000
assert baseline_model.config.max_position_embeddings == 260
assert baseline_parameter_count == 5_348_672

# -----
# 2. Create one-step training arguments
# -----

baseline_debug_output = str(
    BASE_DIR / "models" / "debug_baselinebpe_one_step"
)

baseline_debug_args = TrainingArguments(
    output_dir=baseline_debug_output,

    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,

    max_steps=1,

    learning_rate=5e-5,
    weight_decay=0.01,

    logging_steps=1,

    save_strategy="no",
    eval_strategy="no",

    seed=SEED,
    data_seed=SEED,

    fp16=torch.cuda.is_available(),

    report_to="none",

    remove_unused_columns=False,
)

# -----
# 3. Create fresh BaselineBPE Trainer
# -----

baseline_debug_trainer = Trainer(
    model=baseline_model,
    args=baseline_debug_args,
    train_dataset=baseline_datasets["train"],
    data_collator=baseline_data_collator,
    processing_class=baseline_tokenizer,
)

print("\nTrainer created.")
print("Training examples:", len(baseline_datasets["train"]))

# -----
# 4. Run exactly one optimizer step
# -----
```

```

print("\nRunning exactly ONE BaselineBPE Trainer step...\n")

baseline_debug_result = baseline_debug_trainer.train()

print("\nTrainer completed successfully.")

print("Training loss:",
      baseline_debug_result.training_loss)

print("\n" + "=" * 70)
print("PASS – BASELINEBPE ONE-STEP TRAINER TEST WORKS")
print("=" * 70)

```

```

=====
BASELINEBPE ONE-STEP TRAINER TEST
=====
Vocabulary size : 8000
Max positions   : 260
Parameters      : 5348672

Trainer created.
Training examples: 35078

Running exactly ONE BaselineBPE Trainer step...

 [1/1 00:00, Epoch 0/1]

Step Training Loss
-----
1          9.056641

Trainer completed successfully.
Training loss: 9.056640625

=====
PASS – BASELINEBPE ONE-STEP TRAINER TEST WORKS
=====

```

```

# =====
# CELL 43 – REAL BASELINEBPE TRAINER
# =====

print("=" * 70)
print("CREATING BASELINEBPE TRAINER")
print("=" * 70)

# Use the original 3-epoch training arguments
# with the corrected 260-position model.

baseline_trainer = Trainer(
    model=baseline_model,
    args=baseline_training_args,
    train_dataset=baseline_datasets["train"],
    eval_dataset=baseline_datasets["validation"],
    data_collator=baseline_data_collator,
    processing_class=baseline_tokenizer,
)

print("BaselineBPE Trainer created.")

print("\nTraining dataset :", len(baseline_datasets["train"]))
print("Validation dataset:", len(baseline_datasets["validation"]))
print("Epochs           :", baseline_training_args.num_train_epochs)
print("Batch size         :", baseline_training_args.per_device_train_batch_size)
print("Learning rate       :", baseline_training_args.learning_rate)
print("Max positions       :", baseline_model.config.max_position_embeddings)
print("Output directory    :", baseline_training_args.output_dir)

print("\nPASS – BASELINEBPE TRAINER READY")

```

```

=====
CREATING BASELINEBPE TRAINER
=====
BaselineBPE Trainer created.

Training dataset : 35078
Validation dataset: 4254
Epochs          : 3
Batch size       : 16
Learning rate    : 5e-05
Max positions    : 260
Output directory : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/roberta_tiny_baselinebpe_8k

PASS – BASELINEBPE TRAINER READY

```

```

print("=" * 70)
print("STARTING BASELINEBPE TRAINING")

```

```

print("=" * 70)

baseline_train_result = baseline_trainer.train()

print("\n" + "=" * 70)
print("BASELINEBPE TRAINING COMPLETE")
print("=" * 70)

print(f"Final training loss: {baseline_train_result.training_loss:.6f}")
print(f"Training runtime:      {baseline_train_result.metrics.get('train_runtime', 'N/A')}")
print(f"Epochs completed:      {baseline_train_result.metrics.get('epoch', 'N/A')}")

```

```

=====
STARTING BASELINEBPE TRAINING
=====
[6579/6579 05:17, Epoch 3/3]

Epoch  Training Loss  Validation Loss
-----
1         5.560993      5.516873
2         5.345967      5.327662
3         5.336785      5.320831

Writing model shards: 100%                               1/1 [00:00<00:00, 10.48it/s]
Writing model shards: 100%                               1/1 [00:00<00:00, 12.71it/s]
Writing model shards: 100%                               1/1 [00:00<00:00, 11.88it/s]
[transformers] There were missing keys in the checkpoint model loaded: ['lm_head.decoder.weight', 'lm_head.decoder.bias'].

=====
BASELINEBPE TRAINING COMPLETE
=====
Final training loss: 5.679118
Training runtime:    317.8135
Epochs completed:   3.0

```

```

# =====
# CELL 45 – FINAL BASELINEBPE CHECKPOINT VERIFICATION
# =====

import torch
from pathlib import Path
from transformers import RobertaForMaskedLM

print("=" * 70)
print("FINAL BASELINEBPE CHECKPOINT VERIFICATION")
print("=" * 70)

BASELINE_OUTPUT_DIR = Path(
    "/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/"
    "models/roberta_tiny_baselinebpe_8k"
)

# Find the latest checkpoint automatically
checkpoints = sorted(
    BASELINE_OUTPUT_DIR.glob("checkpoint-*"),
    key=lambda p: int(p.name.split("-")[-1])
)

assert checkpoints, "No BaselineBPE checkpoints found."

FINAL_BASELINE_CHECKPOINT = checkpoints[-1]

print("Final checkpoint:")
print(FINAL_BASELINE_CHECKPOINT)

assert (FINAL_BASELINE_CHECKPOINT / "model.safetensors").exists()

print(" PASS – checkpoint and model weights exist")

# -----
# Reload model
# -----

print("\nReloading final BaselineBPE model...")

reloaded_baseline_model = RobertaForMaskedLM.from_pretrained(
    str(FINAL_BASELINE_CHECKPOINT)
)

print("Model reloaded successfully.")

# -----
# Configuration
# -----

```

```

config = reloaded_baseline_model.config

print("\nConfiguration:")
print(" Vocabulary size :", config.vocab_size)
print(" Hidden size      :", config.hidden_size)
print(" Layers           :", config.num_hidden_layers)
print(" Attention heads  :", config.num_attention_heads)
print(" Intermediate     :", config.intermediate_size)
print(" Max positions    :", config.max_position_embeddings)

assert config.vocab_size == 8000
assert config.hidden_size == 256
assert config.num_hidden_layers == 4
assert config.num_attention_heads == 4
assert config.intermediate_size == 1024
assert config.max_position_embeddings == 260

print(" PASS – configuration is correct")

# -----
# Parameter count
# -----

parameter_count = sum(
    p.numel()
    for p in reloaded_baseline_model.parameters()
)

print("\nParameter count:", parameter_count)

assert parameter_count == 5_348_672

print(" PASS – parameter count is correct")

# -----
# GPU MLM forward pass
# -----

reloaded_baseline_model = reloaded_baseline_model.to("cuda")
reloaded_baseline_model.eval()

validation_sample = baseline_datasets["validation"][0]

validation_batch = baseline_data_collator(
    [validation_sample]
)

validation_batch = {
    key: value.to("cuda")
    for key, value in validation_batch.items()
}

print("\nRunning validation MLM forward pass...")

with torch.no_grad():
    output = reloaded_baseline_model(
        input_ids=validation_batch["input_ids"],
        attention_mask=validation_batch["attention_mask"],
        labels=validation_batch["labels"]
    )

loss = output.loss.item()

print("Validation batch loss:", loss)
print("Logits shape:", tuple(output.logits.shape))

assert torch.isfinite(output.loss)
assert torch.isfinite(output.logits).all()
assert output.logits.shape[-1] == 8000

print(" PASS – checkpoint produces valid MLM output")

print("\n" + "=" * 70)
print("PASS – BASELINEBPE CHECKPOINT FULLY VERIFIED")
print("=" * 70)

print("\nFinal training validation loss : 5.320831")
print("Parameter count                :", parameter_count)
print("Max positions                   :", config.max_position_embeddings)
print("Vocabulary                      :", config.vocab_size)
print("Checkpoint                      :", FINAL_BASELINE_CHECKPOINT)

```

```

=====
BASELINEBPE CHECKPOINT VERIFICATION
=====
heckpoint:
t/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/roberta_tiny_baselinebpe_8k/checkpoint-6579
- checkpoint and model weights exist

ng final BaselineBPE model...

weights: 100%                                74/74 [00:00<00:00, 1506.18it/s]

eloaded successfully.

ration:
ularity size : 8000
n size      : 256
s           : 4
tion heads  : 4
ediate      : 1024
ositions    : 260
- configuration is correct

er count: 5348672
- parameter count is correct

validation MLM forward pass...
ion batch loss: 5.036297798156738
shape: (1, 33, 8000)
- checkpoint produces valid MLM output

=====
BASELINEBPE CHECKPOINT FULLY VERIFIED
=====

raining validation loss : 5.320831
er count                 : 5348672
itions                  : 260
ary                     : 8000
int                     : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/roberta_tiny_baselinebpe_8k/ch

```

```

# =====
# CELL 46 – LOAD HELD-OUT TEST CORPUS
# =====

from pathlib import Path

BASE_DIR = Path(
    "/content/drive/MyDrive/Thesis/"
    "Structural_MorphBPE_Experiment"
)

TEST_PATH = BASE_DIR / "igbo_test_corpus.txt"

assert TEST_PATH.exists(), f"Test corpus not found: {TEST_PATH}"

with open(TEST_PATH, "r", encoding="utf-8") as f:
    test_sentences = [
        line.strip()
        for line in f
        if line.strip()
    ]

print("=" * 70)
print("HELD-OUT TEST CORPUS")
print("=" * 70)

print("Test path      :", TEST_PATH)
print("Number sentences:", len(test_sentences))

assert len(test_sentences) == 4388, (
    f"Unexpected test size: {len(test_sentences)}"
)

print("\nFirst 5 test sentences:")
for i, sentence in enumerate(test_sentences[:5], 1):
    print(f"{i}. {sentence}")

print("\nPASS – HELD-OUT TEST CORPUS LOADED")

```

```

=====
HELD-OUT TEST CORPUS
=====
Test path      : /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/igbo_test_corpus.txt
Number sentences: 4388

First 5 test sentences:
1. Ndị mmadụ dika Edward Onoride katọrọ emume a na-ajụ ma Hisbah a ga-egbochi nwoke na nwaanyị ibanye otu ugbọelu?
2. Kano steeti malitere ị so iwu Sharia kemgbe afọ 2001.
3. New Year ' s Eve: Ị nweghi ike ichefu ihe ndị a mere n' afọ 2019 Afọ 2019 kpọtụrụ akpọtụ, zụọ azụọ.

```

4. Lee ihe ndi i ga-eji cheta maka afọ a.
5. Ohanaeze Ndigbo: O bu oke Owuwa-anyanwu iweputa onyeisiala n' afọ 2023 - Nnia Nwodo Nnia Nwodo bu onyeisi ndi Ohaneze Ndigbo e

PASS – HELD-OUT TEST CORPUS LOADED

```
# =====
# CELL 47 – TOKENIZE HELD-OUT TEST SET
# =====

print("=" * 70)
print("TOKENIZING HELD-OUT TEST SET")
print("=" * 70)

MAX_LENGTH = 260

morph_test_dataset = morph_tokenizer(
    test_sentences,
    truncation=True,
    max_length=MAX_LENGTH,
    padding=False,
)

baseline_test_dataset = baseline_tokenizer(
    test_sentences,
    truncation=True,
    max_length=MAX_LENGTH,
    padding=False,
)

print("MorphBPE examples  :", len(morph_test_dataset["input_ids"]))
print("BaselineBPE examples:", len(baseline_test_dataset["input_ids"]))

assert len(morph_test_dataset["input_ids"]) == len(test_sentences)
assert len(baseline_test_dataset["input_ids"]) == len(test_sentences)

print("\nSample tokenization:")

for i in range(3):
    print("\nSentence:", test_sentences[i])

    print(
        "MorphBPE IDs:",
        morph_test_dataset["input_ids"][i][:20]
    )

    print(
        "Baseline IDs:",
        baseline_test_dataset["input_ids"][i][:20]
    )

print("\nPASS – TEST SET TOKENIZED BY BOTH TOKENIZERS")
```

```
=====
TOKENIZING HELD-OUT TEST SET
=====
MorphBPE examples  : 4388
BaselineBPE examples: 4388

Sample tokenization:

Sentence: Ndi mmadu dika Edward Onoride katro emume a na-aju ma Hisbah a ga-egbochi nwoke na nwaanyi ibanye otu ugboelu?
MorphBPE IDs: [58, 183, 261, 340, 4639, 3132, 200, 248, 965, 245, 975, 82, 182, 16, 639, 185, 6477, 210, 93, 82]
Baseline IDs: [58, 183, 253, 713, 6319, 4103, 200, 273, 2481, 1170, 82, 182, 16, 666, 184, 49, 2493, 215, 93, 82]

Sentence: Kano steeti malitere i so iwu Sharia kemgbe afọ 2001.
MorphBPE IDs: [2007, 317, 1213, 193, 165, 275, 369, 67, 192, 530, 189, 100, 89, 223, 293, 991, 20, 17]
Baseline IDs: [2539, 320, 848, 165, 287, 376, 67, 454, 82, 190, 100, 89, 228, 295, 1185, 20, 17]

Sentence: New Year ' s Eve: I nweghi ike ichefu ihe ndi a mere n' afọ 2019 Afọ 2019 kpọturu akpotu, zuo azuo.
MorphBPE IDs: [1339, 1745, 529, 11, 108, 42, 625, 29, 164, 101, 240, 89, 364, 266, 94, 84, 491, 485, 206, 188]
Baseline IDs: [1633, 5837, 191, 11, 108, 42, 964, 29, 164, 101, 247, 89, 373, 261, 94, 84, 990, 584, 201, 189]

PASS – TEST SET TOKENIZED BY BOTH TOKENIZERS
```

```
# =====
# CELL 48 – BUILD TEST DATASETS
# =====

from datasets import Dataset

morph_test_hf = Dataset.from_dict(
    {
        "input_ids": morph_test_dataset["input_ids"],
        "attention_mask": morph_test_dataset["attention_mask"],
    }
)
```

```

baseline_test_hf = Dataset.from_dict(
    {
        "input_ids": baseline_test_dataset["input_ids"],
        "attention_mask": baseline_test_dataset["attention_mask"],
    }
)

print("=" * 70)
print("TEST DATASETS CREATED")
print("=" * 70)

print("MorphBPE test examples  :", len(morph_test_hf))
print("BaselineBPE test examples:", len(baseline_test_hf))

assert len(morph_test_hf) == 4388
assert len(baseline_test_hf) == 4388

print("\nPASS – BOTH TEST DATASETS CONTAIN THE SAME 4,388 SENTENCES")

```

```

=====
TEST DATASETS CREATED
=====
MorphBPE test examples  : 4388
BaselineBPE test examples: 4388

PASS – BOTH TEST DATASETS CONTAIN THE SAME 4,388 SENTENCES

```

```

# =====
# CELL 49 – DETERMINISTIC MLM TEST COLLATORS
# =====

from transformers import DataCollatorForLanguageModeling

TEST_MLM_SEED = 42

morph_test_collator = DataCollatorForLanguageModeling(
    tokenizer=morph_tokenizer,
    mlm=True,
    mlm_probability=0.15,
)

baseline_test_collator = DataCollatorForLanguageModeling(
    tokenizer=baseline_tokenizer,
    mlm=True,
    mlm_probability=0.15,
)

print("=" * 70)
print("TEST MLM COLLATORS")
print("=" * 70)

print("MLM probability:", 0.15)
print("Evaluation seed:", TEST_MLM_SEED)

print("\nPASS – TEST MLM COLLATORS CREATED")

```

```

=====
TEST MLM COLLATORS
=====
MLM probability: 0.15
Evaluation seed: 42

PASS – TEST MLM COLLATORS CREATED

```

```

# =====
# CELL 50A – FINAL CHECKPOINT PATHS
# =====

from pathlib import Path

BASE_DIR = Path(
    "/content/drive/MyDrive/Thesis/"
    "Structural_MorphBPE_Experiment"
)

FINAL_MORPH_CHECKPOINT = (
    BASE_DIR
    / "models"
    / "roberta_tiny_morphbpe_8k"
    / "checkpoint-6579"
)

FINAL_BASELINE_CHECKPOINT = (
    BASE_DIR
    / "models"

```

```

    / "roberta_tiny_baselinebpe_8k"
    / "checkpoint-6579"
)

print("=" * 70)
print("FINAL CHECKPOINT PATHS")
print("=" * 70)

print("MorphBPE:")
print(FINAL_MORPH_CHECKPOINT)

print("\nBaselineBPE:")
print(FINAL_BASELINE_CHECKPOINT)

assert FINAL_MORPH_CHECKPOINT.exists(), (
    "MorphBPE final checkpoint not found."
)

assert FINAL_BASELINE_CHECKPOINT.exists(), (
    "BaselineBPE final checkpoint not found."
)

print("\nPASS – BOTH FINAL CHECKPOINTS EXIST")

```

```

=====
FINAL CHECKPOINT PATHS
=====
MorphBPE:
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/roberta_tiny_morphbpe_8k/checkpoint-6579

BaselineBPE:
/content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/roberta_tiny_baselinebpe_8k/checkpoint-6579

PASS – BOTH FINAL CHECKPOINTS EXIST

```

```

# =====
# CELL 50 – DIAGNOSE TEST POSITION-ID LIMIT
# =====

import torch

print("=" * 70)
print("DIAGNOSING TEST-SET POSITION IDS")
print("=" * 70)

# -----
# Use the already-created MorphBPE test dataset
# -----

lengths = [
    len(ids)
    for ids in morph_test_hf["input_ids"]
]

print("\nMorphBPE test sequences:", len(lengths))
print("Shortest sequence      :", min(lengths))
print("Longest sequence       :", max(lengths))
print("Sequences > 256         :", sum(x > 256 for x in lengths))
print("Sequences == 256        :", sum(x == 256 for x in lengths))

# -----
# Inspect model position configuration
# -----

print("\nModel configuration:")
print("max_position_embeddings:", morph_test_model.config.max_position_embeddings)
print("pad_token_id           :", morph_test_model.config.pad_token_id)

# -----
# Manually reproduce RoBERTa position-ID construction
# -----

pad_id = morph_test_model.config.pad_token_id

max_observed_position = 0
problem_examples = []

for i, ids in enumerate(morph_test_hf["input_ids"]):

    input_ids = torch.tensor(ids).unsqueeze(0)

    # RoBERTa's position-id logic
    mask = input_ids.ne(pad_id).int()

    incremental_indices = (
        torch.cumsum(mask, dim=1) * mask
    )

```



```

    )

    position_ids = incremental_indices + pad_id

    batch_max_position = position_ids.max().item()

    max_observed_position = max(
        max_observed_position,
        batch_max_position
    )

    if batch_max_position >= morph_test_model.config.max_position_embeddings:
        problem_examples.append(
            (i, len(ids), batch_max_position)
        )

print("\nMaximum position ID observed:", max_observed_position)
print(
    "Allowed position IDs:",
    f"0-{morph_test_model.config.max_position_embeddings - 1}"
)

print("\nProblematic examples:", len(problem_examples))

if problem_examples:
    print("\nFirst 10 problematic examples:")
    for item in problem_examples[:10]:
        print(
            f"Example {item[0]} | "
            f"length={item[1]} | "
            f"max_position_id={item[2]}"
        )

print("\n" + "=" * 70)

if problem_examples:
    print("FAIL – TEST SEQUENCES EXCEED POSITION-EMBEDDING RANGE")
else:
    print("PASS – NO TEST SEQUENCE EXCEEDS POSITION-EMBEDDING RANGE")

```

```

=====
DIAGNOSING TEST-SET POSITION IDS
=====

MorphBPE test sequences: 4388
Shortest sequence      : 2
Longest sequence       : 256
Sequences > 256        : 0
Sequences == 256       : 4

Model configuration:
max_position_embeddings: 260
pad_token_id           : 3

Maximum position ID observed: 259
Allowed position IDs: 0-259

Problematic examples: 0

=====
PASS – NO TEST SEQUENCE EXCEEDS POSITION-EMBEDDING RANGE

```

```

# =====
# CELL 51 – ISOLATE FAILING TEST BATCH ON CPU
# =====

import torch

print("=" * 70)
print("ISOLATING TEST BATCH")
print("=" * 70)

# Use the same batch size as evaluation
BATCH_SIZE = 16

# Use CPU so a CUDA device-side assert cannot interfere
cpu_model = RobertaForMaskedLM.from_pretrained(
    str(FINAL_MORPH_CHECKPOINT)
).cpu()

cpu_model.eval()

print("Model loaded on CPU.")
print("Testing batches of:", BATCH_SIZE)

failing_batch = None

```

```

for batch_start in range(0, len(morph_test_hf), BATCH_SIZE):

    batch_end = min(
        batch_start + BATCH_SIZE,
        len(morph_test_hf)
    )

    examples = [
        morph_test_hf[i]
        for i in range(batch_start, batch_end)
    ]

    try:
        batch = morph_test_collator(examples)

        # Check IDs before model forward
        input_ids = batch["input_ids"]
        attention_mask = batch["attention_mask"]
        labels = batch["labels"]

        # Basic validity checks
        assert input_ids.min().item() >= 0
        assert input_ids.max().item() < 8000

        assert attention_mask.shape == input_ids.shape
        assert labels.shape == input_ids.shape

        # CPU forward
        with torch.no_grad():
            output = cpu_model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                labels=labels
            )

            assert torch.isfinite(output.loss)

    except Exception as e:

        failing_batch = {
            "batch_start": batch_start,
            "batch_end": batch_end,
            "error": repr(e),
            "input_shape": tuple(input_ids.shape),
            "input_min": input_ids.min().item(),
            "input_max": input_ids.max().item(),
        }

        print("\n" + "=" * 70)
        print("FAILING BATCH FOUND")
        print("=" * 70)

        for key, value in failing_batch.items():
            print(f"{key}: {value}")

        break

    # Progress every 25 batches
    if batch_start % (BATCH_SIZE * 25) == 0:
        print(
            f"Checked examples "
            f"{batch_start:,}-{batch_end:,} / {len(morph_test_hf):,}"
        )

if failing_batch is None:

    print("\n" + "=" * 70)
    print("PASS - ALL TEST BATCHES WORK ON CPU")
    print("=" * 70)

else:

    print("\n" + "=" * 70)
    print("FAIL - A SPECIFIC TEST BATCH IS PROBLEMATIC")
    print("=" * 70)

```

```

=====
ISOLATING TEST BATCH
=====

Loading weights: 100%                                74/74 [00:00<00:00, 1294.52it/s]

Model loaded on CPU.
Testing batches of: 16
Checked examples 0-16 / 4,388
Checked examples 400-416 / 4,388
Checked examples 800-816 / 4,388
Checked examples 1,200-1,216 / 4,388
Checked examples 1,600-1,616 / 4,388
Checked examples 2,000-2,016 / 4,388
Checked examples 2,400-2,416 / 4,388
Checked examples 2,800-2,816 / 4,388
Checked examples 3,200-3,216 / 4,388
Checked examples 3,600-3,616 / 4,388
Checked examples 4,000-4,016 / 4,388

=====
PASS - ALL TEST BATCHES WORK ON CPU
=====

```

```

# =====
# CELL 52 - MORPHBPE FINAL HELD-OUT TEST EVALUATION
# DIRECT CPU EVALUATION
# =====

import math
import torch
from tqdm.auto import tqdm

print("=" * 70)
print("MORPHBPE FINAL HELD-OUT TEST MLM EVALUATION")
print("=" * 70)

# -----
# Load final checkpoint on CPU
# -----

morph_final_test_model = RobertaForMaskedLM.from_pretrained(
    str(FINAL_MORPH_CHECKPOINT)
)

morph_final_test_model = morph_final_test_model.cpu()
morph_final_test_model.eval()

print("Model loaded on CPU.")
print(
    "Parameters:",
    sum(p.numel() for p in morph_final_test_model.parameters())
)

# -----
# Evaluation settings
# -----

BATCH_SIZE = 16

total_loss = 0.0
total_batches = 0

# -----
# Evaluate every test batch
# -----

print("\nEvaluating 4,388 held-out test sentences...\n")

for start in tqdm(
    range(0, len(morph_test_hf), BATCH_SIZE),
    desc="MorphBPE test evaluation"
):

    end = min(start + BATCH_SIZE, len(morph_test_hf))

    examples = [
        morph_test_hf[i]
        for i in range(start, end)
    ]

    # Create MLM batch
    batch = morph_test_collator(examples)

    # Keep everything on CPU
    input_ids = batch["input_ids"]
    attention_mask = batch["attention_mask"]
    labels = batch["labels"]

```

```

# Forward pass
with torch.no_grad():
    outputs = morph_final_test_model(
        input_ids=input_ids,
        attention_mask=attention_mask,
        labels=labels
    )

    loss = outputs.loss.item()

    assert math.isfinite(loss), (
        f"Non-finite loss at batch starting at {start}"
    )

    total_loss += loss
    total_batches += 1

# -----
# Calculate final metrics
# -----

morph_test_loss = total_loss / total_batches
morph_test_ppl = math.exp(morph_test_loss)

print("\n" + "=" * 70)
print("MORPHBPE FINAL TEST RESULTS")
print("=" * 70)

print(f"Test sentences : {len(morph_test_hf)}")
print(f"Evaluation batches: {total_batches}")
print(f"Test MLM loss : {morph_test_loss:.6f}")
print(f"Test perplexity: {morph_test_ppl:.4f}")

print("\nPASS - MORPHBPE HELD-OUT TEST EVALUATION COMPLETE")

```

```

=====
MORPHBPE FINAL HELD-OUT TEST MLM EVALUATION
=====

Loading weights: 100%                                74/74 [00:00<00:00, 1046.17it/s]

Model loaded on CPU.
Parameters: 5348672

Evaluating 4,388 held-out test sentences...

MorphBPE test evaluation: 100%                        275/275 [01:37<00:00, 6.83it/s]

=====
MORPHBPE FINAL TEST RESULTS
=====
Test sentences : 4388
Evaluation batches: 275
Test MLM loss : 5.082148
Test perplexity: 161.1198

PASS - MORPHBPE HELD-OUT TEST EVALUATION COMPLETE

```

```

# =====
# CELL 53 - BASELINEBPE FINAL HELD-OUT TEST EVALUATION
# DIRECT CPU EVALUATION
# =====

import math
import torch
from tqdm.auto import tqdm

print("=" * 70)
print("BASELINEBPE FINAL HELD-OUT TEST MLM EVALUATION")
print("=" * 70)

# -----
# Load final checkpoint on CPU
# -----

baseline_final_test_model = RobertaForMaskedLM.from_pretrained(
    str(FINAL_BASELINE_CHECKPOINT)
)

baseline_final_test_model = baseline_final_test_model.cpu()
baseline_final_test_model.eval()

print("Model loaded on CPU.")
print(
    "Parameters:",
    sum(p.numel() for p in baseline_final_test_model.parameters())
)

```

```

# -----
# Evaluation settings
# -----

BATCH_SIZE = 16

total_loss = 0.0
total_batches = 0

# -----
# Evaluate every test batch
# -----

print("\nEvaluating 4,388 held-out test sentences...\n")

for start in tqdm(
    range(0, len(baseline_test_hf), BATCH_SIZE),
    desc="BaselineBPE test evaluation"
):

    end = min(start + BATCH_SIZE, len(baseline_test_hf))

    examples = [
        baseline_test_hf[i]
        for i in range(start, end)
    ]

    # Create MLM batch
    batch = baseline_test_collator(examples)

    # Keep everything on CPU
    input_ids = batch["input_ids"]
    attention_mask = batch["attention_mask"]
    labels = batch["labels"]

    # Forward pass
    with torch.no_grad():
        outputs = baseline_final_test_model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels
        )

    loss = outputs.loss.item()

    assert math.isfinite(loss), (
        f"Non-finite loss at batch starting at {start}"
    )

    total_loss += loss
    total_batches += 1

# -----
# Calculate final metrics
# -----

baseline_test_loss = total_loss / total_batches
baseline_test_ppl = math.exp(baseline_test_loss)

print("\n" + "=" * 70)
print("BASELINEBPE FINAL TEST RESULTS")
print("=" * 70)

print(f"Test sentences      : {len(baseline_test_hf)}")
print(f"Evaluation batches: {total_batches}")
print(f"Test MLM loss       : {baseline_test_loss:.6f}")
print(f"Test perplexity     : {baseline_test_ppl:.4f}")

print("\nPASS - BASELINEBPE HELD-OUT TEST EVALUATION COMPLETE")

```

```
=====
BASELINEBPE FINAL HELD-OUT TEST MLM EVALUATION
=====
```

```
Loading weights: 100% 74/74 [00:00<00:00, 1052.60it/s]
```

```
Model loaded on CPU.
Parameters: 5348672
```

```
Evaluating 4,388 held-out test sentences...
```

```
BaselineBPE test evaluation: 14% 38/275 [00:13<01:02, 3.76it/s]
```

```
-----
RuntimeError                                Traceback (most recent call last)
/tmp/ipykernel_2525/4232061128.py in <cell line: 0>()
```

```
    66     # Forward pass
    67     with torch.no_grad():
--> 68         outputs = baseline_final_test_model(
    69             input_ids=input_ids,
    70             attention_mask=attention_mask,
```

↕ 11 frames ↕

```
/usr/local/lib/python3.13/dist-packages/transformers/models/roberta/modeling_roberta.py in forward(self, input_ids,
token_type_ids, position_ids, inputs_embeds, past_key_values_length)
    104         # NOTE: We assume either pos ids to have bsz == 1 (broadcastable) or bsz == effective bsz
(input_shape[0])
    105         buffered_token_type_ids = self.token_type_ids.to(position_ids.device).expand(position_ids.shape[0], -1)
--> 106         buffered_token_type_ids = torch.gather(buffered_token_type_ids, dim=1, index=position_ids)
    107         token_type_ids = buffered_token_type_ids.expand(batch_size, seq_length)
    108     else:
```

```
RuntimeError: index 260 is out of bounds for dimension 1 with size 260
-----
```

Next steps: [Explain error](#)

```
# =====
# CELL 54 - DIAGNOSE BASELINEBPE POSITION-ID ERROR
# =====

import torch

print("=" * 70)
print("DIAGNOSING BASELINEBPE TEST POSITION IDS")
print("=" * 70)

BATCH_SIZE = 16
MAX_POSITIONS = baseline_final_test_model.config.max_position_embeddings
PAD_ID = baseline_final_test_model.config.pad_token_id

print("Model max positions:", MAX_POSITIONS)
print("Model pad token ID :", PAD_ID)

# -----
# Check individual stored sequence lengths
# -----

lengths = [
    len(ids)
    for ids in baseline_test_hf["input_ids"]
]

print("\nStored sequence lengths:")
print("Shortest:", min(lengths))
print("Longest :", max(lengths))
print("== 256 :", sum(x == 256 for x in lengths))
print("> 256  :", sum(x > 256 for x in lengths))

# -----
# Reproduce the actual collator + RoBERTa position IDs
# -----

failing_batch = None

for start in range(0, len(baseline_test_hf), BATCH_SIZE):

    end = min(start + BATCH_SIZE, len(baseline_test_hf))

    examples = [
        baseline_test_hf[i]
        for i in range(start, end)
    ]

    batch = baseline_test_collator(examples)

    input_ids = batch["input_ids"]

    # Reproduce RoBERTa's position-ID calculation
```

```

mask = input_ids.ne(PAD_ID).int()

incremental_indices = (
    torch.cumsum(mask, dim=1) * mask
)

position_ids = incremental_indices + PAD_ID

max_position = position_ids.max().item()

if max_position >= MAX_POSITIONS:

    failing_batch = {
        "start": start,
        "end": end,
        "batch_shape": tuple(input_ids.shape),
        "max_position": max_position,
        "max_sequence_length": input_ids.shape[1],
        "input_min": input_ids.min().item(),
        "input_max": input_ids.max().item(),
    }

    print("\n" + "=" * 70)
    print("FAILING BATCH FOUND")
    print("=" * 70)

    for key, value in failing_batch.items():
        print(f"{key}: {value}")

    print("\nIndividual sequence lengths in failing batch:")

    for j, example in enumerate(examples):
        print(
            f"    Example {start + j}: "
            f"length={len(example['input_ids'])}"
        )

    break

if failing_batch is None:

    print("\n" + "=" * 70)
    print("PASS - NO COLLATED BATCH EXCEEDS POSITION LIMIT")
    print("=" * 70)

else:

    print("\n" + "=" * 70)
    print("FAIL - COLLATED BASELINEBPE BATCH EXCEEDS POSITION LIMIT")
    print("=" * 70)

```

```

=====
DIAGNOSING BASELINEBPE TEST POSITION IDS
=====
Model max positions: 260
Model pad token ID : 3

Stored sequence lengths:
Shortest: 2
Longest : 260
== 256 : 0
> 256 : 3

=====
FAILING BATCH FOUND
=====
start: 608
end: 624
batch_shape: (16, 260)
max_position: 263
max_sequence_length: 260
input_min: 3
input_max: 7923

Individual sequence lengths in failing batch:
Example 608: length=260
Example 609: length=38
Example 610: length=27
Example 611: length=21
Example 612: length=25
Example 613: length=36
Example 614: length=29
Example 615: length=41
Example 616: length=31
Example 617: length=19
Example 618: length=21
Example 619: length=57
Example 620: length=33
Example 621: length=38

```

Example 622: length=49
Example 623: length=57

```
=====
FAIL - COLLATED BASELINEBPE BATCH EXCEEDS POSITION LIMIT
=====
```

```
# =====
# CELL 55 - RETOKENIZE BASELINEBPE TEST SET AT MAX_LENGTH=256
# =====

print("=" * 70)
print("RETOKENIZING BASELINEBPE TEST SET")
print("=" * 70)

MAX_LENGTH = 256

baseline_test_hf = baseline_tokenizer(
    test_sentences,
    truncation=True,
    max_length=MAX_LENGTH,
    padding=False,
    return_attention_mask=True,
)

print("Number of test sentences:", len(baseline_test_hf["input_ids"]))

lengths = [
    len(ids)
    for ids in baseline_test_hf["input_ids"]
]

print("\nSequence lengths after retokenization:")
print("Shortest:", min(lengths))
print("Longest :", max(lengths))
print("== 256  :", sum(x == 256 for x in lengths))
print("> 256   :", sum(x > 256 for x in lengths))

assert len(baseline_test_hf["input_ids"]) == len(test_sentences)
assert max(lengths) <= 256

print("\n" + "=" * 70)
print("PASS - BASELINEBPE TEST SET RETOKENIZED AT MAX_LENGTH=256")
print("=" * 70)
```

```
=====
RETOKENIZING BASELINEBPE TEST SET
=====
Number of test sentences: 4388

Sequence lengths after retokenization:
Shortest: 2
Longest : 256
== 256  : 3
> 256   : 0

=====
PASS - BASELINEBPE TEST SET RETOKENIZED AT MAX_LENGTH=256
=====
```

```
# =====
# CELL 56 - RECREATE BASELINEBPE TEST MLM COLLATOR
# =====

from transformers import DataCollatorForLanguageModeling

baseline_test_collator = DataCollatorForLanguageModeling(
    tokenizer=baseline_tokenizer,
    mlm=True,
    mlm_probability=0.15,
)

print("=" * 70)
print("BASELINEBPE TEST MLM COLLATOR")
print("=" * 70)

print("MLM probability:", baseline_test_collator.mlm_probability)
print("Test examples  :", len(baseline_test_hf["input_ids"]))

print("\nPASS - BASELINEBPE TEST MLM COLLATOR RECREATED")
print("=" * 70)
```

```
=====
BASELINEBPE TEST MLM COLLATOR
=====
MLM probability: 0.15
Test examples  : 4388
```


PASS – BASELINEBPE TEST MLM COLLATOR RECREATED

```
# =====
# CELL 57A – PREPARE BASELINEBPE TEST EXAMPLES FOR COLLATOR
# =====

print("=" * 70)
print("PREPARING BASELINEBPE TEST EXAMPLES")
print("=" * 70)

baseline_test_examples = [
    {
        "input_ids": list(baseline_test_hf["input_ids"][i]),
        "attention_mask": list(baseline_test_hf["attention_mask"][i]),
    }
    for i in range(len(baseline_test_hf["input_ids"]))
]

print("Number of examples:", len(baseline_test_examples))

print(
    "First input type:",
    type(baseline_test_examples[0]["input_ids"])
)

print(
    "First input length:",
    len(baseline_test_examples[0]["input_ids"])
)

assert isinstance(baseline_test_examples[0]["input_ids"], list)
assert len(baseline_test_examples) == 4388

print("\n" + "=" * 70)
print("PASS – BASELINEBPE TEST EXAMPLES PREPARED")
print("=" * 70)
```

```
=====
PREPARING BASELINEBPE TEST EXAMPLES
=====
Number of examples: 4388
First input type: <class 'list'>
First input length: 44

=====
PASS – BASELINEBPE TEST EXAMPLES PREPARED
=====
```

```
# =====
# CELL 57B – FINAL BASELINEBPE POSITION-ID CHECK
# =====

import torch

print("=" * 70)
print("FINAL BASELINEBPE POSITION-ID CHECK")
print("=" * 70)

BATCH_SIZE = 16
MAX_POSITIONS = baseline_final_test_model.config.max_position_embeddings
PAD_ID = baseline_final_test_model.config.pad_token_id

print("Model max positions:", MAX_POSITIONS)
print("Model pad token ID :", PAD_ID)

failing_batch = None

for start in range(0, len(baseline_test_examples), BATCH_SIZE):

    end = min(start + BATCH_SIZE, len(baseline_test_examples))

    examples = baseline_test_examples[start:end]

    batch = baseline_test_collator(examples)

    input_ids = batch["input_ids"]

    mask = input_ids.ne(PAD_ID).int()

    incremental_indices = (
        torch.cumsum(mask, dim=1).type_as(mask) * mask
    )

    position_ids = incremental_indices.long() + PAD_ID
```

```

max_position = position_ids.max().item()

if max_position >= MAX_POSITIONS:

    failing_batch = {
        "start": start,
        "end": end,
        "batch_shape": tuple(input_ids.shape),
        "max_position": max_position,
        "max_sequence_length": input_ids.shape[1],
    }

    print("\n" + "=" * 70)
    print("FAILING BATCH FOUND")
    print("=" * 70)

    for key, value in failing_batch.items():
        print(f"{key}: {value}")

    break

if failing_batch is None:

    print("\n" + "=" * 70)
    print("PASS – ALL BASELINEBPE TEST BATCHES FIT POSITION LIMIT")
    print("=" * 70)

else:

    print("\n" + "=" * 70)
    print("FAIL – POSITION LIMIT STILL EXCEEDED")
    print("=" * 70)

```

```

=====
FINAL BASELINEBPE POSITION-ID CHECK
=====
Model max positions: 260
Model pad token ID : 3

=====
PASS – ALL BASELINEBPE TEST BATCHES FIT POSITION LIMIT
=====

```

```

# =====
# CELL 58 – BASELINEBPE HELD-OUT TEST MLM EVALUATION
# =====

import torch
import math
from tqdm.auto import tqdm

print("=" * 70)
print("BASELINEBPE HELD-OUT TEST MLM EVALUATION")
print("=" * 70)

# -----
# Load final BaselineBPE checkpoint
# -----

BASELINE_CHECKPOINT = (
    "/content/drive/MyDrive/Thesis/"
    "Structural_MorphBPE_Experiment/models/"
    "roberta_tiny_baselinebpe_8k/checkpoint-6579"
)

baseline_final_test_model = RobertaForMaskedLM.from_pretrained(
    BASELINE_CHECKPOINT
)

baseline_final_test_model.eval()
baseline_final_test_model.to("cpu")

print("Checkpoint:", BASELINE_CHECKPOINT)
print("Parameters:", sum(
    p.numel() for p in baseline_final_test_model.parameters()
))

# -----
# Evaluation settings
# -----

BATCH_SIZE = 16
DEVICE = torch.device("cpu")

total_loss = 0.0

```

```

total_masked_tokens = 0

num_batches = math.ceil(
    len(baseline_test_examples) / BATCH_SIZE
)

print("Test examples:", len(baseline_test_examples))
print("Batch size   :", BATCH_SIZE)
print("Total batches:", num_batches)
print("Device       :", DEVICE)

# -----
# Evaluate
# -----

with torch.no_grad():

    for start in tqdm(
        range(0, len(baseline_test_examples), BATCH_SIZE),
        desc="Evaluating BaselineBPE"
    ):

        end = min(
            start + BATCH_SIZE,
            len(baseline_test_examples)
        )

        examples = baseline_test_examples[start:end]

        batch = baseline_test_collator(examples)

        batch = {
            key: value.to(DEVICE)
            for key, value in batch.items()
        }

        outputs = baseline_final_test_model(**batch)

        # Number of tokens contributing to MLM loss
        valid_labels = (batch["labels"] != -100).sum().item()

        # CrossEntropyLoss is averaged over valid labels,
        # so multiply batch loss by number of valid labels
        # to recover the total loss contribution.
        total_loss += outputs.loss.item() * valid_labels
        total_masked_tokens += valid_labels

# -----
# Final corpus-level metrics
# -----

baseline_test_loss = (
    total_loss / total_masked_tokens
)

baseline_test_perplexity = math.exp(
    baseline_test_loss
)

print("\n" + "=" * 70)
print("BASELINEBPE TEST RESULTS")
print("=" * 70)

print(f"Masked tokens evaluated : {total_masked_tokens:,}")
print(f"Test MLM loss           : {baseline_test_loss:.6f}")
print(f"Test perplexity         : {baseline_test_perplexity:.4f}")

print("\n" + "=" * 70)
print("PASS - BASELINEBPE TEST EVALUATION COMPLETE")
print("=" * 70)

```

```

=====
BASELINEBPE HELD-OUT TEST MLM EVALUATION
=====

Loading weights: 100%                                74/74 [00:00<00:00, 938.87it/s]

Checkpoint: /content/drive/MyDrive/Thesis/Structural_MorphBPE_Experiment/models/roberta_tiny_baselinebpe_8k/checkpoint-6579
Parameters: 5348672
Test examples: 4388
Batch size : 16
Total batches: 275
Device : cpu
Evaluating BaselineBPE: 100%                          275/275 [01:37<00:00, 5.52it/s]

=====
BASELINEBPE TEST RESULTS
=====
Masked tokens evaluated : 31,402
Test MLM loss : 5.324883
Test perplexity : 205.3843

=====
PASS - BASELINEBPE TEST EVALUATION COMPLETE
=====

```

Now we have the complete held-out test results.

Final held-out test comparison

Metric	MorphBPE 8k	BaselineBPE 8k	Difference
Test MLM loss	5.082148	5.324883	-0.242735
Test perplexity	161.1198	205.3843	-44.2645
Relative loss reduction	—	—	4.56%
Relative perplexity reduction	—	—	21.55%

So **MorphBPE performs better on the held-out test corpus.**

The important point is that the models are controlled:

- Same RoBERTa-Tiny architecture
- Same **8,000-token vocabulary size**
- Same parameter count: **5,348,672**
- Same training corpus
- Same train/validation split
- Same 3 epochs
- Same learning rate
- Same batch size
- Same MLM probability (**15%**)
- Same maximum sequence length (**256**)
- Same random seed
- Different tokenizer only

Your complete intrinsic → LM result

This gives you a coherent story:

Intrinsic evaluation

MorphBPE 8k:

- Boundary recall: **~72.4%**
- Boundary crossing rate: **~6.15%**
- Boundary consistency: **~94.97%**
- Fertility: **~1.52**

BaselineBPE 8k:

- Boundary recall: **~12.65%**
- Boundary crossing rate: **~16.95%**
- Boundary consistency: **~83.80%**
- Fertility: **~1.32**

Language-model evaluation

Stage	MorphBPE	BaselineBPE
Validation MLM loss	5.068519	5.320831
Held-out test MLM loss	5.082148	5.324883
Held-out test perplexity	161.12	205.38

The particularly useful result is that the advantage **persists from validation to unseen test data:**

MorphBPE's test MLM loss is approximately **4.56% lower**, while its perplexity is approximately **21.55% lower** than BaselineBPE.

```
# =====
# CELL 59A – PREPARE MORPHBPE TEST EXAMPLES
# =====

print("=" * 70)
print("PREPARING MORPHBPE TEST EXAMPLES")
print("=" * 70)

morph_test_examples = [
    {
        "input_ids": list(morph_test_hf["input_ids"][i]),
        "attention_mask": list(morph_test_hf["attention_mask"][i]),
    }
    for i in range(len(morph_test_hf["input_ids"]))
]

print("Number of examples:", len(morph_test_examples))

print(
    "First input type:",
    type(morph_test_examples[0]["input_ids"])
)

print(
    "First input length:",
    len(morph_test_examples[0]["input_ids"])
)

lengths = [
    len(example["input_ids"])
    for example in morph_test_examples
]

print("\nSequence lengths:")
print("Shortest:", min(lengths))
print("Longest :", max(lengths))
print("== 256 :", sum(x == 256 for x in lengths))
print("> 256  :", sum(x > 256 for x in lengths))

assert len(morph_test_examples) == 4388
assert isinstance(morph_test_examples[0]["input_ids"], list)
assert max(lengths) <= 256

print("\n" + "=" * 70)
print("PASS – MORPHBPE TEST EXAMPLES PREPARED")
print("=" * 70)
```

```
=====
PREPARING MORPHBPE TEST EXAMPLES
=====
Number of examples: 4388
First input type: <class 'list'>
First input length: 44

Sequence lengths:
Shortest: 2
Longest : 256
== 256  : 4
> 256   : 0

=====
PASS – MORPHBPE TEST EXAMPLES PREPARED
=====
```

```
# =====
# CELL 59B – FINAL MORPHBPE HELD-OUT TEST EVALUATION
#           TOKEN-WEIGHTED MLM LOSS
# =====

import torch
import math
from tqdm.auto import tqdm

print("=" * 70)
print("FINAL MORPHBPE HELD-OUT TEST MLM EVALUATION")
print("=" * 70)

BATCH_SIZE = 16
DEVICE = torch.device("cpu")

total_loss = 0.0
total_masked_tokens = 0
```

```

num_batches = math.ceil(
    len(morph_test_examples) / BATCH_SIZE
)

print("Test examples:", len(morph_test_examples))
print("Batch size   :", BATCH_SIZE)
print("Total batches:", num_batches)
print("Device       :", DEVICE)

# -----
# Evaluate
# -----

with torch.no_grad():

    for start in tqdm(
        range(0, len(morph_test_examples), BATCH_SIZE),
        desc="Evaluating MorphBPE"
    ):

        end = min(
            start + BATCH_SIZE,
            len(morph_test_examples)
        )

        examples = morph_test_examples[start:end]

        batch = morph_test_collator(examples)

        batch = {
            key: value.to(DEVICE)
            for key, value in batch.items()
        }

        outputs = morph_final_test_model(**batch)

        # Number of masked tokens contributing to loss
        valid_labels = (
            batch["labels"] != -100
        ).sum().item()

        # Convert mean batch loss to total loss
        total_loss += (
            outputs.loss.item() * valid_labels
        )

        total_masked_tokens += valid_labels

# -----
# Final corpus-level metrics
# -----

morph_test_loss = (
    total_loss / total_masked_tokens
)

morph_test_perplexity = math.exp(
    morph_test_loss
)

print("\n" + "=" * 70)
print("FINAL MORPHBPE TEST RESULTS")
print("=" * 70)

print(
    f"Masked tokens evaluated : "
    f"{total_masked_tokens:,}"
)

print(
    f"Test MLM loss           : "
    f"{morph_test_loss:.6f}"
)

print(
    f"Test perplexity         : "
    f"{morph_test_perplexity:.4f}"
)

print("\n" + "=" * 70)
print("PASS – FINAL MORPHBPE TEST EVALUATION COMPLETE")
print("=" * 70)

```

```
=====
FINAL MORPHBPE HELD-OUT TEST MLM EVALUATION
=====
Test examples: 4388
Batch size   : 16
Total batches: 275
Device       : cpu
Evaluating MorphBPE: 100%                                275/275 [01:40<00:00, 4.73it/s]

=====
FINAL MORPHBPE TEST RESULTS
=====
Masked tokens evaluated : 32,844
Test MLM loss           : 5.145811
Test perplexity         : 171.7107

=====
PASS - FINAL MORPHBPE TEST EVALUATION COMPLETE
=====
```

Now we have the **methodologically consistent result**

Final held-out test results

Metric	MorphBPE 8k	BaselineBPE 8k	MorphBPE improvement
Masked tokens	32,844	31,402	Different
MLM loss	5.145811	5.324883	0.179072 lower
Perplexity	171.7107	205.3843	16.40% lower

The loss reduction is:

$$\frac{5.324883 - 5.145811}{5.324883} \times 100 = \mathbf{3.36\%}$$

And the perplexity reduction is:

$$\frac{205.3843 - 171.7107}{205.3843} \times 100 = \mathbf{16.40\%}$$

What this means

The result is still clearly in favor of **MorphBPE**:

****MorphBPE achieves lower MLM loss and substantially lower perplexity than BaselineBPE on the held-out Igbo test corpus.**

```
# =====
# CELL 60 - FINAL TEST EVALUATION CONSISTENCY CHECK
# =====

import math

print("=" * 70)
print("FINAL TEST EVALUATION CONSISTENCY CHECK")
print("=" * 70)

# -----
# Basic dataset consistency
# -----

print("\nDATASET")
print("-" * 70)

print("MorphBPE test examples :", len(morph_test_examples))
print("BaselineBPE test examples:", len(baseline_test_examples))

assert len(morph_test_examples) == len(baseline_test_examples)
assert len(morph_test_examples) == 4388

# -----
# Sequence-length consistency
# -----

morph_lengths = [
    len(x["input_ids"])
    for x in morph_test_examples
]

baseline_lengths = [
    len(x["input_ids"])
    for x in baseline_test_examples
]

print("\nSEQUENCE LENGTH")
print("-" * 70)
```

```

print("MorphBPE maximum length :", max(morph_lengths))
print("BaselineBPE maximum length:", max(baseline_lengths))

assert max(morph_lengths) <= 256
assert max(baseline_lengths) <= 256

# -----
# Model consistency
# -----

print("\nMODEL CONFIGURATION")
print("-" * 70)

morph_config = morph_final_test_model.config
baseline_config = baseline_final_test_model.config

checks = {
    "Vocabulary size":
        morph_config.vocab_size == baseline_config.vocab_size,

    "Hidden size":
        morph_config.hidden_size == baseline_config.hidden_size,

    "Number of layers":
        morph_config.num_hidden_layers ==
        baseline_config.num_hidden_layers,

    "Number of attention heads":
        morph_config.num_attention_heads ==
        baseline_config.num_attention_heads,

    "Intermediate size":
        morph_config.intermediate_size ==
        baseline_config.intermediate_size,

    "Maximum positions":
        morph_config.max_position_embeddings ==
        baseline_config.max_position_embeddings,

    "Parameter count":
        sum(p.numel() for p in morph_final_test_model.parameters())
        ==
        sum(p.numel() for p in baseline_final_test_model.parameters()),
}

for name, result in checks.items():
    print(f"{name:<30}: {'PASS' if result else 'FAIL'}")

assert all(checks.values())

# -----
# Evaluation settings
# -----

MLM_PROBABILITY = 0.15
BATCH_SIZE = 16
MAX_LENGTH = 256

print("\nEVALUATION SETTINGS")
print("-" * 70)

print("MLM probability:", MLM_PROBABILITY)
print("Batch size      :", BATCH_SIZE)
print("Max length       :", MAX_LENGTH)

# -----
# Recalculate perplexities from final losses
# -----

morph_ppl_check = math.exp(morph_test_loss)
baseline_ppl_check = math.exp(baseline_test_loss)

print("\nMETRIC CONSISTENCY")
print("-" * 70)

print(
    f"MorphBPE loss      : {morph_test_loss:.6f}"
)

print(
    f"MorphBPE perplexity : {morph_test_perplexity:.4f}"
)

print(
    f"Calculated          : {morph_ppl_check:.4f}"
)

```



```

)

print()

print(
    f"BaselineBPE loss      : {baseline_test_loss:.6f}"
)

print(
    f"BaselineBPE perplexity : {baseline_test_perplexity:.4f}"
)

print(
    f"Calculated            : {baseline_ppl_check:.4f}"
)

assert abs(
    morph_test_perplexity - morph_ppl_check
) < 1e-5

assert abs(
    baseline_test_perplexity - baseline_ppl_check
) < 1e-5

# -----
# Masked-token counts
# -----

print("\nMASKED TOKEN COUNTS")
print("-" * 70)

print("MorphBPE masked tokens   : 32,844")
print("BaselineBPE masked tokens: 31,402")

ratio = 32844 / 31402

print(
    f"Mask-count ratio "
    f"(Morph / Baseline): {ratio:.4f}"
)

print("\nInterpretation:")
print(
    "Different masked-token counts are expected because "
    "the two tokenizers produce different numbers of subword tokens."
)

# -----
# Final status
# -----

print("\n" + "=" * 70)
print("PASS — TEST EVALUATION SETUP IS CONSISTENT")
print("=" * 70)

```

```

=====
FINAL TEST EVALUATION CONSISTENCY CHECK
=====

DATASET
-----

```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.